强化学习（Reinforcement Learning）的数学原理

作者：牛牧霖

核心逻辑：强化学习的本质是“智能体（Agent）在环境（Environment）中，通过‘尝试-反馈’不断学习，找到最优行为策略”，类比人类“摸爬滚打”的学习过程（比如学骑车、学游戏）。

第一章：强化学习的背景

1.1 强化学习的核心场景（通俗理解）

举3个零基础能听懂的例子，建立直观认知：

- 例子1：老鼠走迷宫——老鼠（智能体）在迷宫（环境）中，每次选择一个方向（动作），走到死胡同（负面反馈）就回头，走到出口（正面反馈，比如奶酪）就记住路线，反复尝试后找到最短路线。
- 例子2：AlphaGo下棋——AlphaGo（智能体）和人类/自己（环境）下棋，每走一步（动作），根据棋局变化（状态变化）获得反馈（比如这步走得好/差），反复训练后，学会“赢棋的最优走法”。
- 例子3：小孩学走路——小孩（智能体）在地面（环境）中，尝试迈步（动作），站稳了（小奖励）、摔倒了（惩罚），反复尝试后，学会稳定走路（最优行为）。

核心共性：**没有“标准答案”，只有“反馈信号”**，智能体通过“尝试→获得反馈→调整行为”，逐步找到最优策略（能获得最大反馈的一系列动作）。

1.2 和监督学习、无监督学习的区别（避免混淆）

学习类型	核心特点（通俗说）	类比场景
监督学习	有“标准答案”（标签），比如“猫的图片→标签‘猫’”，学习“输入→标准答案”的映射	老师给你做题，每道题都有标准答案，你学的是“题目→答案”
无监督学习	没有标签，只有数据，学习数据的“内在规律”（比如聚类）	给你一堆水果，没有标签，你自己找出“苹果、橘子、香蕉”的分类规律
强化学习	没有标准答案，只有“奖励/惩罚”（反馈），学习“动作→最大反馈”的策略	你玩游戏，没有说明书，只有“得分（奖励）”和“扣分（惩罚）”，自己摸索赢游戏的方法

1.3 数学基础

核心假设（马尔可夫性质）： “未来只取决于现在，而与过去无关。” 只要我们知道现在的状态，就不需要翻看历史日志来预测未来。这个假设极大地简化了数学计算的复杂度。

核心公式$P(S_{t+1}|S_t) = P(S_{t+1}|S_1, S_2, \ldots, S_t)$

第二章：基础符号定义

> 说明：所有符号均采用强化学习领域**最通用的标注**，每个符号都配“通俗解释+场景对应”；后续所有公式，均基于这些符号推导，不会新增未定义的符号。本文中状态动作奖励五元组(s, a, s', r, a')**和**$(s_t, a_t, s_{t+1}, r_t, a_{t+1})$是等价的

2.1 核心实体符号

- **智能体（Agent）**：符号无特定标注，通常直接称为“智能体”，是学习的主体（比如老鼠、AlphaGo、小孩）。
- **环境（Environment）**：符号无特定标注，通常直接称为“环境”，是智能体所处的场景（比如迷宫、棋局、地面），会对智能体的动作做出反馈。强化学习中的环境一般就是一个**马尔可夫决策过程（Markov Decision Process，MDP）**

2.2 状态、动作相关符号

- **状态（State）**：
  - 符号：s（单个状态）、S（状态空间，所有可能状态的集合）
    - 通俗解释：智能体在环境中的“当前处境”，比如“老鼠在迷宫的A点”“棋局中黑棋剩3个子”“小孩站在原地没动”。
    - 补充：s_t 表示“第t步时，智能体的状态”（t是时间步，比如t=1表示第一步，t=2表示第二步）；s_0 通常是“初始状态”（比如老鼠从迷宫入口出发，入口就是 s_0）。
- **动作（Action）**：
  - 符号：a（单个动作）、A（动作空间，所有可能动作的集合）

- 通俗解释：智能体在当前状态下，能做出的“行为”，比如“老鼠向左走”“AlphaGo下在棋盘的(3,3)位置”“小孩迈右脚”。
- 补充：a_t 表示“第t步时，智能体选择的动作”即“s_t状态下选择的动作”；动作空间 A 是“当前状态下所有能做的动作”，比如老鼠在A点，能向左、向右、向前走，那么 $A=\{向左,向右,向前\}$。

- **状态转移（State Transition）**：
  - 符号：$P(s'|s,a)$等价于$P(s_{t+1}|s_t,a_t)$
  - 通俗解释：智能体在“状态 s 下，选择动作 a 后，转移到新状态 s' 的概率”（s' 读作s-prime，代表下一个状态）。
  - 例子：老鼠在状态 s （A点），选择动作 a （向左走），有80%概率走到 s' （B点），20%概率撞到墙（仍在A点），那么 $P(B|A,向左)=0.8$， $P(A|A,向左)=0.2$。
  - 关键：转移概率是“环境的属性”，智能体无法控制，只能去“学习”或“适应”（比如老鼠慢慢知道，向左走大概率到B点）。
- **轨迹（Trajectory）**
  - 符号：τ
  - 轨迹是智能体从“起点”到“终点”的一条完整“路线”$\tau=(s_0,a_0,r_0,s_1,a_1,r_1,\ldots,s_t,a_t,r_t,\ldots,s_T)$

2.3 反馈相关符号（奖励、回报）

- **奖励（Reward）**：
  - 符号：r （单个奖励）、r_t （第t步的奖励）等价于$R(s_t,a_t,s_{t+1})$
    - 通俗解释：智能体在“状态 s_t 下，选择动作 a_t (转移到状态s_{t+1})后，环境给出的“即时反馈”，是强化学习的“核心驱动力”——正面反馈（奖励为正）、负面反馈（奖励为负）、无反馈（奖励为0）。
    - 例子：老鼠在A点向左走（到B点），给1分奖励（$r_t=1$）；如果撞到墙（仍在A点），给-1分惩罚（$r_t=-1$）；如果原地不动，给0分（$r_t=0$）。
    - 关键：奖励是“即时的”，只对应“当前一步”的动作好坏。
- **回报（Return）**：
  - 符号：G_t （第t步及之后的总回报）
    - 通俗解释：“长期反馈”，是第t步的即时奖励 r_t，加上第t+1步的奖励 r_{t+1}，加上第t+2步的奖励 r_{t+2}，直到学习结束（终止状态）的总和。
    - 补充：终止状态（Terminal State）：符号 s_T （T是最后一步），表示“学习结束的状态”，比如老鼠走到迷宫出口、AlphaGo赢/输棋、小孩学会走路站稳。终止状态后，没有后续动作和奖励。

> G_t是随机采样的一条路径所计算的具体数值，$V^\pi(s)$和$Q^\pi(s,a)$都是对G_t的期望

2.4 策略、价值相关符号

- **策略函数（Policy Function）**：
  - 符号：$\pi(a|s)$
  - 通俗解释：智能体的“行为规则”——在状态 s 下，选择动作 a 的概率（或确定的动作）。
  - 例子：老鼠在A点（状态 s），策略 $\pi(向左|A)=0.7$， $\pi(向右|A)=0.2$， $\pi(向前|A)=0.1$，表示“老鼠在A点时，70%概率向左走，20%向右，10%向前”。
  - 关键：强化学习的“目标”，就是找到“最优策略” π^* （星号表示最优），使得总回报 G_t 最大。
  - **参数化策略**
    - 符号：$\pi_\theta(a\mid s)$
    - 通俗解释：带**可学习参数** θ 的策略（θ 一般是神经网络的权重、偏置）。
    - 关键：我们不再直接找 π^*，而是**学习最优参数** θ^*，让 $\pi_{\theta^*}(a\mid s)$ 成为最优策略
  - 策略熵（policy entropy）
    - 符号：$S[\pi_\theta](s_t)$或$\mathcal{H}(\pi_\theta(\cdot|s_t))$
    - 公式：$\mathcal{H}(\pi_\theta(\cdot|s_t))=\mathbb{E}_{a\sim\pi_\theta}[\log\pi_\theta(\cdot\mid s_t)]$
    - 含义：用于衡量策略（Policy）在给定状态下选择动作的**不确定性**或**随机性**。它是平衡 **探索（Exploration）** 与 **利用（Exploitation）** 的关键工具。
- **价值函数（Value Function）**：
  - 符号1（状态价值函数）：$V^\pi(s)$，表示“在策略 π 下，处于状态 s 时，未来能获得的期望总回报”（期望：因为状态转移有概率，所以取平均回报）。
  - 符号2（动作价值函数）：$Q^\pi(s,a)$，表示“在策略 π 下，处于状态 s、选择动作 a 后，未来能获得的期望总回报”。
  - 通俗解释：价值函数是“智能体的判断标准”——$V^\pi(s)$ 判断“这个状态好不好”， $Q^\pi(s,a)$ 判断“在这个状态下，做这个动作好不好”。
  - 例子：$V^\pi(A)=5$， $V^\pi(B)=3$，表示“按照策略 π，老鼠在A点的未来总回报期望是5，比在B点（3）好”；$Q^\pi(A,向左)=6$， $Q^\pi(A,向右)=4$，表示“在A点，向左走比向右走的未来回报更高”。

- **参数化价值函数**
  - 状态价值参数化：$V_\theta(s)$
  - 动作价值参数化：$Q_\theta(s,a)$
  - 通俗解释：用 带参数θ的函数（如神经网络）去近似真实价值 V_π、Q_π。
  - 关键：深度强化学习就是通过更新θ，让 V_θ、Q_θ越来越准。
- **优势函数（Advantage Function）**
  - 符号：$A^\pi(s,a)$
  - 通俗解释："在状态 s下，选动作a，比这个状态的平均水平好多少"是一个相对值，不是绝对值。
  - 公式：$A^\pi(s,a) = Q^\pi(s,a) - V^\pi(s)$
  - 例子：$V^\pi(A) = 5$ （A 点的平均价值）$Q^\pi(A, \text{向左}) = 6$，则 $A^\pi(A, \text{向左}) = 6 - 5 = 1$表示：在 A 点向左走，比平均水平好 1。
  - 关键含义：
    A>0：这个动作比平均好，应该多做
    A<0：这个动作比平均差，应该少做
    A=0：和平均水平一样
  - **参数化优势函数**
    - 符号：$A_\theta(s,a)$
    - 公式：$A_\theta(s,a) = Q_\theta(s,a) - V_\theta(s)$
    - 通俗解释：用参数化的 Q、V 算出的优势，是 PPO、A2C 等算法更新策略的**核心信号**。

2.5 折扣因子

- 符号：γ （gamma，读作"伽马"）
- 取值范围：$0 \le \gamma \le 1$
- 通俗解释："对未来回报的重视程度"——相当于"存钱的利息"，未来的钱（回报）不如现在的钱（回报）值钱，所以要打折扣。
- 细节：
  - $\gamma = 1$：未来回报和现在回报一样重要（比如老鼠只关心"最终能不能到出口"，不管花多少步）；
  - $\gamma = 0$：只关心当前一步的奖励（比如老鼠只在乎"现在走一步能不能得分"，不管未来能不能到出口）；
  - 实际应用：γ 通常取0.9~0.99，既重视当前奖励，也重视未来回报（比如AlphaGo既在乎"当前一步能不能占优"，也在乎"最终能不能赢棋"）。

2.6 折扣状态访问分布（Discounted State Visitation Distribution）

最标准的归一化定义：

$d^\pi(s) = (1-\gamma)\sum_{t=0}^{\infty} \gamma^t P_\pi(s_t = s | s_0 \sim P_0)$

我们把它拆成四个部分来理解：

- $P_\pi(s_t = s | s_0 \sim P_0)$：这是**边缘概率**。它表示如果初始状态 s_0 服从环境的初始分布 P_0，并且智能体一直遵循策略 π 行动，那么在**确切的第 t 个时间步**，智能体恰好处于状态 s 的概率。
- γ^t：这是**折扣因子**。由于强化学习关注的是累积折扣回报，未来的状态对当前目标的影响是衰减的。因此，第 t 步访问到状态 s 的重要性，要乘上 γ^t 进行打折。
- $\sum_{t=0}^{\infty}$：这是**时间维度的累加**。因为智能体在一集（Episode）中可能会多次经过同一个状态 s，我们需要把从 $t=0$ 开始到无限远的时间里，每次经过状态 s 的折扣概率全部加起来。
- $(1-\gamma)$：这是**归一化常数**。因为一个等比数列 $\sum_{t=0}^{\infty} \gamma^t = \frac{1}{1-\gamma}$，乘上 $(1-\gamma)$ 就可以保证所有状态的 $d^\pi(s)$ 加起来正好等于 1（即 $\sum_s d^\pi(s) = 1$），从而让它成为一个严谨的**概率分布**。

折扣状态访问分布反应当智能体使用策略 π 在环境中持续交互时各个状态访问概率（比例）若两个策略相似其对应的折扣状态访问分布也相似。

2.7目标函数

如果把强化学习比作一场"人生修行"，那么**目标函数（Objective Function）**就是你的"人生理想"或"北极星"。它告诉智能体（Agent）：**什么样的行为才是"好"的，我们最终追求的到底是什么？**在 RL 中，目标函数通常是为了量化智能体策略的优劣 。

目标函数的概念一般只出现在策略学习中（价值学习优化的价值函数也可以视为它的目标函数），是策略函数的终极优化目标
$J(\pi) = \mathbb{E}_{\tau \sim \pi}(R(\tau))$

符号汇总表

符号	名称	通俗解释（核心）
s, S	状态、状态空间	当前处境、所有可能的处境
a, A	动作、动作空间	能做的行为、所有可能的行为
s_t, a_t	第t步的状态、动作	第t步的处境、第t步做的行为
τ	轨迹	智能体从“起点”到“终点”的一条完整“路线”
$P(s_{t+1} \mid s_t, a_t)$	状态转移概率	在s_t状态做a_t动作，到s_{t+1}状态的概率
r_t	第t步的奖励	当前一步的即时反馈（好/坏）
G_t	第t步的总回报	第t步及之后的所有奖励总和（长期反馈）
γ	折扣因子	对未来回报的重视程度（0~1）
$\pi(a \mid s)$，$\pi_\theta(a \mid s)$	策略	s状态下，选a动作的规则（概率）
$S[\pi_\theta](s_t)$，$\mathcal{H}(\pi_\theta(\cdot \mid s_t))$	策略熵	衡量策略的随机性
$V^\pi(s)$，$V_\theta(s)$	状态价值函数	策略π下，s状态的未来期望总回报
$Q^\pi(s,a)$，$Q_\theta(s,a)$	动作价值函数	策略π下，s状态选a动作的未来期望总回报
$A^\pi(s,a)$，$A_\theta(s,a)$	优势函数	策略π下,动作a比一般情况下多带来多少期望回报
s_T	终止状态	学习结束的状态（无后续动作/奖励）
$d^\pi(s)$	折扣状态访问分布	当前策略下访问状态的分布
$J(\pi)$	目标函数	策略函数的优化目标

第三章：核心公式推导

3.1 总回报G_t

3.1.1 无折扣回报（γ=1，不考虑未来回报的折扣）

核心逻辑：总回报 = 第t步奖励 + 第t+1步奖励 + ... + 最后一步（终止状态前）的奖励。

公式推导：

$$G_t = r_t + r_{t+1} + r_{t+2} + \ldots + r_{T-1}$$

文字解释：

- G_t：第t步的总回报（从t步开始，到学习结束的所有奖励总和）；
- r_t：第t步的即时奖励（当前一步的反馈）；
- r_{t+1}：第t+1步的奖励（下一步的反馈）；
- r_{T-1}：第T步的奖励（终止状态前的最后一步奖励，因为终止状态 s_T 没有动作，所以没有 r_T ）。

例子（老鼠走迷宫）：

假设老鼠从t=1开始，走到终止状态（出口）用了4步（T=4），各步奖励如下：$r_1 = 1$ （第一步得分）、$r_2 = 2$ （第二步得分）、$r_3 = -1$ （第三步扣分）、$r_4 = 10$ （第四步到出口，大奖）。

则第1步的总回报：$G_1 = r_1 + r_2 + r_3 + r_4 = 1 + 2 - 1 + 10 = 12$；

第2步的总回报：$G_2 = r_2 + r_3 + r_4 = 2 - 1 + 10 = 11$；

第4步的总回报：$G_4 = r_4 = 10$ （只有当前一步奖励，没有后续）。

3.1.2 折扣回报（$\gamma \in$[0,1]，实际应用中最常用）

核心问题：无折扣回报有一个缺陷——“未来的回报和现在的回报一样重要”，但实际场景中，“眼前的奖励更可靠”（比如老鼠更在乎“现在走一步能不能得分”，而不是“10步后能不能到出口”），所以需要加入折扣因子 γ，对未来的回报打折扣。

公式推导：

$G_t = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \ldots + \gamma^{T-t-1} r_{T-1}$

文字解释：

- 和无折扣回报相比，新增了“折扣项” γ^k （k是“未来的步数差”）；
- 第t+1步的回报，折扣项是 γ^1 （步数差为1），第t+2步是 γ^2 （步数差为2），以此类推，步数越靠后，折扣越大，回报的权重越低；
- 当 $\gamma = 1$ 时，折扣回报就等于无折扣回报（ $\gamma^k = 1$ ）；当 $\gamma = 0$ 时， $G_t = r_t$ （只关注当前一步奖励）。

例子（沿用上面的老鼠迷宫，取 $\gamma = 0.9$ ）：

第1步的总回报： $G_1 = r_1 + 0.9 r_2 + 0.9^2 r_3 + 0.9^3 r_4$

代入数值： $G_1 = 1 + 0.9 \times 2 + 0.81 \times (-1) + 0.729 \times 10 \approx 1 + 1.8 - 0.81 + 7.29 = 9.28$ ；

第2步的总回报： $G_2 = r_2 + 0.9 r_3 + 0.9^2 r_4 \approx 2 + 0.9 \times (-1) + 0.81 \times 10 = 2 - 0.9 + 8.1 = 9.2$ ；

对比无折扣回报（ $G_1 = 12$ ），折扣回报更“务实”——未来的大奖（ $r_4 = 10$ ）被打了折扣（0.729倍），权重降低，更符合实际学习逻辑。

3.1.3 回报的递归公式（后续贝尔曼方程推导的关键）

核心逻辑：第t步的总回报 G_t ，可以拆成“第t步的即时奖励 r_t ”加上“第t+1步的总回报 G_{t+1} 的折扣值”。

公式推导（从折扣回报公式推导）：

已知： $G_t = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \ldots + \gamma^{T-t-1} r_{T-1}$

而第t+1步的总回报： $G_{t+1} = r_{t+1} + \gamma r_{t+2} + \ldots + \gamma^{T-t-2} r_{T-1}$

将 G_{t+1} 乘以 γ ，得到： $\gamma G_{t+1} = \gamma r_{t+1} + \gamma^2 r_{t+2} + \ldots + \gamma^{T-t-1} r_{T-1}$

代入 G_t 的公式，可得：

$G_t = r_t + \gamma G_{t+1}$

文字解释：这是一个“递归公式”——当前步的总回报，等于“现在的奖励”加上“下一步总回报的折扣”，非常简洁，却是后续所有核心公式的基础。

例子：沿用上面的老鼠迷宫， $G_4 = 10$ ，则 $G_3 = r_3 + \gamma G_4 = -1 + 0.9 \times 10 = 8$ ； $G_2 = r_2 + \gamma G_3 = 2 + 0.9 \times 8 = 9.2$ （和之前计算一致）； $G_1 = r_1 + \gamma G_2 = 1 + 0.9 \times 9.2 = 9.28$ （完全一致），验证了公式的正确性。

3.2 价值函数的推导（智能体的“判断标准”）

核心前提：价值函数是“期望总回报”，而不是“确定的总回报”——因为状态转移有概率（ $P(s'|s,a)$ ），智能体无法确定“选了动作a后，一定会到哪个状态s'”，所以只能计算“平均下来的总回报”（期望）。

期望的通俗解释：比如抛硬币，正面得10分，反面得0分，抛硬币的期望得分是“0.5×10 + 0.5×0 = 5分”——不是确定能得到5分，而是长期抛下来，平均每次得5分。

3.2.1 状态价值函数$V^{\pi}(s)$的推导

定义回顾： $V^{\pi}(s)$ 是“在策略 π 下，处于状态 s 时，未来能获得的期望总回报”。

公式推导：

第一步：根据定义，状态 s 的价值，是“从状态 s 出发，按照策略 π 选择动作，未来的期望总回报”，也就是 G_t 的期望（因为 G_t 是总回报，期望就是平均总回报）。

$V^{\pi}(s) = \mathbb{E}_{\tau \sim \pi}[G_t | s_t = s]$

文字解释：

- $\mathbb{E}_{\tau \sim \pi}[\cdot]$ ：表示“在策略 π 下的期望”（$\mathbb{E}$是期望的符号，τ是从策略π采样得来的轨迹$\tau = (a_t, s_{t+1}, a_{t+1}, s_{t+2}, \ldots, s_T)$）；
- $[G_t | s_t = s]$ ：表示“在第t步状态为s的条件下，总回报G_t的期望”；
- 整体含义：按照策略π，当智能体处于状态s时，未来能获得的平均总回报，就是状态s的价值。
- 强化学习中G_t不是固定值，因此必须用**期望**衡量平均水平，随机性来自两方面：
  - **策略的随机性**：若π是**随机策略**（$\pi(a \mid s)$表示状态s下选动作a的概率），则动作选择是随机的；
  - **环境的随机性**：即使选同一个动作a，环境也可能跳转到不同的下一个状态s'，并给出不同的即时奖励R（比如游戏中同一操作可能触发不同结果）。（若策略是确定策略、环境是确定性环境，则期望就是确定值，$V^{\pi}(s) = G_t$）

第二步：代入回报的递归公式 $G_t = r_t + \gamma G_{t+1}$ ，得到：

$V^{\pi}(s) = \mathbb{E}_{\tau \sim \pi}[r_t + \gamma G_{t+1} | s_t = s]$

第三步：利用期望的线性性质（$\mathbb{E}[a+b]=\mathbb{E}[a]+\mathbb{E}[b]$，常数可以提到期望外面），拆分期望：

$V^\pi(s)=\mathbb{E}_{\tau\sim\pi}[r_t|s_t=s]+\gamma\mathbb{E}_{\tau\sim\pi}[G_{t+1}|s_t=s]$

第四步：简化第二项期望——当第t步状态为s时，第t+1步的状态 s_{t+1} 由“状态转移概率 $P(s'|s,a)$ ”和“策略 $\pi(a|s)$ ”决定，所以 $\mathbb{E}_{\tau\sim\pi}[G_{t+1}|s_t=s]=\mathbb{E}_{\tau\sim\pi}[V^\pi(s_{t+1})|s_t=s]$ （因为 $V^\pi(s_{t+1})$ 就是 s_{t+1} 状态的期望总回报）。

最终简化后的状态价值函数（结合策略和状态转移概率）：

$V^\pi(s)=\sum_a\pi(a|s)\left[\sum_{s'}P(s'|s,a)\left(r(s,a,s')+\gamma V^\pi(s')\right)\right]=\mathbb{E}_{a\sim\pi(\cdot|s)}\left[\mathbb{E}_{s'\sim P(\cdot|s,a)}\left[r+\gamma V^\pi(s')\right]\right]$

文字解释：

- 外层求和 $\sum_a$ ：对“当前状态s下，所有可能的动作a”求和（因为策略$\pi(a|s)$是选每个动作的概率，需要遍历所有动作）；
- $\pi(a|s)$ ：策略——选动作a的概率；
- 内层求和 $\sum_{s'}$ ：对“选动作a后，所有可能的下一个状态s'”求和（因为状态转移有概率，需要遍历所有可能的下一个状态）；
- $P(s'|s,a)$ ：状态转移概率——选动作a后，到状态s'的概率；
- $r(s,a,s')$ ：在状态s选动作a，转移到s'的即时奖励（和之前的 r_t 含义一致，只是更明确标注了状态和动作的关系）；
- $\gamma V^\pi(s')$ ：下一个状态s'的价值，乘以折扣因子（未来价值的折扣）。

3.2.2 动作价值函数$Q^\pi(s,a)$的推导

定义回顾：$Q^\pi(s,a)$ 是“在策略 π 下，处于状态 s 、选择动作 a 后，未来能获得的期望总回报”。

核心区别：状态价值函数 $V^\pi(s)$ 是“状态好不好”，动作价值函数 $Q^\pi(s,a)$ 是“在这个状态下，做这个动作好不好”——比状态价值更具体，直接指导动作选择。

公式推导：

第一步：根据定义，动作价值是“在状态s选动作a后，总回报G_t的期望”：

$Q^\pi(s,a)=\mathbb{E}_{\tau\sim\pi}[G_t|s_t=s,a_t=a]$

文字解释：按照策略π，当智能体处于状态s、且选择了动作a时，未来能获得的平均总回报，就是这个“状态-动作对”的价值。

第二步：同样代入回报的递归公式 $G_t=r_t+\gamma G_{t+1}$ ，并拆分期望：

$Q^\pi(s,a)=\mathbb{E}_{\tau\sim\pi}[r_t+\gamma G_{t+1}|s_t=s,a_t=a]$

$Q^\pi(s,a)=\mathbb{E}_{\tau\sim\pi}[r_t|s_t=s,a_t=a]+\gamma\mathbb{E}_{\tau\sim\pi}[G_{t+1}|s_t=s,a_t=a]$

第三步：简化第二项期望——选了动作a后，下一个状态s'由状态转移概率 $P(s'|s,a)$ 决定，后续按照策略π执行，所以 $\mathbb{E}_{\tau\sim\pi}[G_{t+1}|s_t=s,a_t=a]=\sum_{s'}P(s'|s,a)V^\pi(s')$ （下一个状态s'的价值期望）。

最终简化后的动作价值函数：

$Q^\pi(s,a)=\sum_{s'}P(s'|s,a)\left(r(s,a,s')+\gamma V^\pi(s')\right)=\mathbb{E}_{s'\sim P(\cdot|s,a)}\left[r(s,a,s')+\gamma V^\pi(s')\right]$

文字解释（通俗拆解）：

- 求和 $\sum_{s'}$ ：对“选动作a后，所有可能的下一个状态s'”求和；
- 每一项都是“转移到s'的概率 ×（转移的即时奖励 + 下一个状态s'的价值折扣）”；
- 和状态价值函数相比，少了“对动作a的求和”——因为动作a已经确定了（Q函数的输入就是s和a）。

补充：状态价值函数和动作价值函数的关系

状态s的价值，等于“在状态s下，所有可能动作a的Q值，乘以选这个动作的概率（策略π），再求和”——因为状态价值是“所有可能动作的平均Q值”。

公式：$V^\pi(s)=\sum_a\pi(a|s)Q^\pi(s,a)=\mathbb{E}_{a\sim\pi(\cdot|s)}Q^\pi(s,a)$

3.3 贝尔曼（Bellman）方程推导

核心意义：贝尔曼方程是“价值函数的递归关系式”，揭示了“当前状态价值（或动作价值）”和“未来状态价值（或动作价值）”之间的关系——是后续所有强化学习算法（Q-learning、SARSA等）的理论基础。

分为“贝尔曼期望方程”（基于当前策略π）和“贝尔曼最优方程”（基于最优策略π^*）。

3.3.1 贝尔曼期望方程（基于当前策略π）

本质：将前面推导的价值函数，整理成“递归形式”——当前价值 = 即时奖励 + 未来价值的折扣期望。

1. **状态价值函数的贝尔曼期望方程**

回顾状态价值函数的最终形式：

$V^\pi(s) = \sum_a \pi(a|s) \left[\sum_{s'} P(s'|s,a) \left(R(s,a,s') + \gamma V^\pi(s')\right)\right]$

结合动作价值函数 $Q^\pi(s,a) = \sum_{s'} P(s'|s,a) \left(R(s,a,s') + \gamma V^\pi(s')\right)$ ，代入上式，得到简化版贝尔曼期望方程：

$V^\pi(s) = \sum_a \pi(a|s) Q^\pi(s,a)$

再将Q函数代入，得到完整的递归形式：

- $V^\pi(s) = \mathbb{E}_{\tau\sim\pi}[r_t + \gamma V^\pi(s_{t+1})|s_t = s]$

文字解释：当前状态s的价值，等于“按照策略π执行，获得的即时奖励r_t的期望”，加上“下一个状态s_{t+1}的价值$V^\pi(s_{t+1})$的折扣期望”——当前价值依赖于未来价值，这就是递归的核心。

2. **动作价值函数的贝尔曼期望方程**

回顾动作价值函数的最终形式：

$Q^\pi(s,a) = \sum_{s'} P(s'|s,a) \left(R(s,a,s') + \gamma V^\pi(s')\right)$

结合状态价值和动作价值的关系 $V^\pi(s') = \sum_{a'} \pi(a'|s') Q^\pi(s',a')$ （下一个状态s'的价值，是s'下所有动作a'的Q值加权和），代入上式，得到：

$Q^\pi(s,a) = \sum_{s'} P(s'|s,a) \left(R(s,a,s') + \gamma \sum_{a'} \pi(a'|s') Q^\pi(s',a')\right)$

简化版（期望形式）：

- $Q^\pi(s,a) = \mathbb{E}_{\tau\sim\pi}[r_t + \gamma Q^\pi(s_{t+1},a_{t+1})|s_t = s, a_t = a]$

文字解释：当前“状态-动作对”(s,a)的价值，等于“即时奖励r_t的期望”，加上“下一个‘状态-动作对’(s_{t+1},a_{t+1})的价值$Q^\pi(s_{t+1},a_{t+1})$的折扣期望”。

3. **动作价值和状态价值的关系**

$V^\pi(s) = \sum_{a\in A} \pi(a \mid s) Q^\pi(s,a) = \mathbb{E}_{a\sim\pi(\cdot|s)} Q^\pi(s,a)$
$Q^\pi(s,a) = \sum_{s'\in S} P(s' \mid s,a)[R(s,a,s') + \gamma V^\pi(s')] = \mathbb{E}_{s'\sim P(\cdot|s,a)}[R(s,a,s') + \gamma V^\pi(s')]$

3.3.2 贝尔曼最优方程（基于最优策略π*）

核心前提：最优策略 π^* 是“能让所有状态的价值都最大的策略”，即 $\pi^* = argmax_\pi V^\pi(s) = argmax_\pi Q^\pi(s,a)$

核心思想

最优状态的价值 = 所有可能动作中即时奖励与未来价值之和的最大值，体现了最优性原理：最优策略的任何子策略也必须是最优的。

最优状态价值函数 $V^*(s)$

表示从状态 s 出发，遵循最优策略 π* 时的最大期望累积折扣奖励：

$V^*(s) = \max_\pi V^\pi(s)$

对应的贝尔曼最优方程：

$V^*(s) = \max_{a\in A} \sum_{s'\in S} P(s' \mid s,a) \left[R(s,a,s') + \gamma V^*(s')\right] = \max_{a\in A} \mathbb{E}_{s'\sim P(\cdot|s,a)} \left[R(s,a,s') + \gamma V^*(s')\right]$

最优动作价值函数 $Q^*(s,a)$

表示从状态 s 出发，采取动作 a后再遵循最优策略时的最大期望累积折扣奖励：

$Q^*(s,a) = \max_\pi Q^\pi(s,a)$

对应的贝尔曼最优方程：

$Q^*(s,a) = \sum_{s'\in S} P(s' \mid s,a) \left[R(s,a,s') + \gamma \max_{a'\in A} Q^*(s',a')\right] = \mathbb{E}_{s'\sim P(\cdot|s,a)} \left[R(s,a,s') + \gamma \max_{a'\in A} Q^*(s',a')\right]$

最优价值函数间的关系

$V^*(s) = \max_{a\in A} Q^*(s,a)$

$Q^*(s,a) = \sum_{s'\in S} P(s' \mid s,a) \left[R(s,a,s') + \gamma V^*(s')\right] = \mathbb{E}_{s'\sim P(\cdot|s,a)} \left[R(s,a,s') + \gamma V^*(s')\right]$

3.4策略梯度定理

策略梯度定理出现的真正原因是：

> 动作是从分布中“采样”的
> 采样操作不可微

你不能直接写：$\nabla_\theta Q(s, a(\theta))$来优化策略网络$\pi_\theta$

策略梯度定理就是面对上述情况下，建立参数θ与Q函数的联系

3.4.1策略梯度的定义

我们的目标是计算期望回报对策略参数的梯度：

$\nabla_\theta J(\theta) = \nabla_\theta \mathbb{E}_{\tau\sim\pi_\theta}[R(\tau)]$

直接对期望求导需要处理积分/求和，这里使用**对数导数技巧（Log-Derivative Trick）**来简化计算。

3.4.2对数导数技巧

对于任意可微概率分布 $p(x)$ 和函数 $f(x)$，有：

$\nabla_\theta \mathbb{E}_{x\sim p_\theta(x)}[f(x)] = \mathbb{E}_{x\sim p_\theta(x)}[f(x)\cdot\nabla_\theta \log p_\theta(x)]$

证明：

$$\begin{aligned}
\nabla_\theta \mathbb{E}[f(x)] &= \nabla_\theta \int p_\theta(x) f(x) dx \\
&= \int \nabla_\theta p_\theta(x) f(x) dx \\
&= \int p_\theta(x) \cdot \frac{\nabla_\theta p_\theta(x)}{p_\theta(x)} \cdot f(x) dx \\
&= \int p_\theta(x) \cdot \nabla_\theta \log p_\theta(x) \cdot f(x) dx \\
&= \mathbb{E}_{x\sim p_\theta(x)}[f(x)\cdot\nabla_\theta \log p_\theta(x)]
\end{aligned}$$

> 对数导数技巧$\nabla_\theta \log \pi = \frac{\nabla_\theta \pi}{\pi}$

将此技巧应用到我们的问题中，令 $x = \tau$，$f(x) = R(\tau)$，$p_\theta(x) = \pi_\theta(\tau)$，则：

$\nabla_\theta J(\theta) = \mathbb{E}_{\tau\sim\pi_\theta}[R(\tau)\cdot\nabla_\theta \log \pi_\theta(\tau)]$

3.4.3轨迹概率的分解

一条轨迹τ 的概率可以分解为初始状态分布、各步策略和环境转移概率的乘积：

$\pi_\theta(\tau) = p(s_0)\prod_{t=0}^{T-1}[\pi_\theta(a_t|s_t)\cdot p(s_{t+1}|s_t, a_t)]$

对其取对数：

$\log \pi_\theta(\tau) = \log p(s_0) + \sum_{t=0}^{T-1}[\log \pi_\theta(a_t|s_t) + \log p(s_{t+1}|s_t, a_t)]$

对 θ 求梯度时，注意到 $p(s_0)$ 和 $p(s_{t+1}|s_t, a_t)$ 与策略参数 θ 无关，因此它们的梯度为 0，可得：

$\nabla_\theta \log \pi_\theta(\tau) = \sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta(a_t|s_t)$

3.4.4代入梯度表达式

将轨迹对数梯度的分解式代入策略梯度公式：

$\nabla_\theta J(\theta) = \mathbb{E}_{\tau\sim\pi_\theta}\left[R(\tau)\cdot\sum_{t=0}^{T-1}\nabla_\theta \log \pi_\theta(a_t|s_t)\right]$

交换求和与期望的顺序：

$\nabla_\theta J(\theta) = \sum_{t=0}^{T-1}\mathbb{E}_{\tau\sim\pi_\theta}[R(\tau)\cdot\nabla_\theta \log \pi_\theta(a_t|s_t)]$

3.4.5利用因果性简化

这里的关键是**因果性**：在时刻 t 采取的动作 a_t 只会影响 t 及之后的回报，对 t 之前的回报没有影响。因此，我们可以将总回报 $R(\tau)$ 拆分为从时刻 t 开始的折扣回报 G_t：

$$R(\tau) = \sum_{k=0}^{T-1}\gamma^k r_k = \sum_{k=0}^{t-1}\gamma^k r_k + \sum_{k=t}^{T-1}\gamma^k r_k$$

带入后有：

$$
\begin{aligned}
\nabla_\theta J(\theta) &= \mathbb{E}_{\tau\sim\pi_\theta}\left[\sum_{t=0}^{T-1}\nabla_\theta\log\pi_\theta(a_t|s_t)\cdot\sum_{k=0}^{t-1}\gamma^k r_k\right]+\mathbb{E}_{\tau\sim\pi_\theta}\left[\sum_{t=0}^{T-1}\nabla_\theta\log\pi_\theta(a_t|s_t)\cdot\sum_{k=t}^{T-1}\gamma^k r_k\right]\\
&= \mathbb{E}_{\tau\sim\pi_\theta}\left[\sum_{t=0}^{T-1}\nabla_\theta\log\pi_\theta(a_t|s_t)\cdot\sum_{k=t}^{T-1}\gamma^k r_k\right]\\
&= \mathbb{E}_{\tau\sim\pi_\theta}\left[\sum_{t=0}^{T-1}\nabla_\theta\log\pi_\theta(a_t|s_t)\cdot\gamma^t G_t\right]
\end{aligned}
$$

> 因为$\sum_{k=0}^{t-1}\gamma^k r_k$只依赖于过去的状态和动作与现在动作$a_t$无关，所以
>
> $$\mathbb{E}_{\tau\sim\pi_\theta}\left[\sum_{t=0}^{T-1}\nabla_\theta\log\pi_\theta(a_t|s_t)\cdot\sum_{k=0}^{t-1}\gamma^k r_k\right]=\sum_{t=0}^{T-1}\sum_{k=0}^{t-1}\gamma^k r_k\cdot\mathbb{E}_{\tau\sim\pi_\theta}\left[\nabla_\theta\log\pi_\theta(a_t|s_t)\right]$$
>
> $$
> \begin{aligned}
> \mathbb{E}_{\tau\sim\pi_\theta}\left[\nabla_\theta\log\pi_\theta(a_t|s_t)\right] &= \mathbb{E}_{s_t\sim\pi_\theta}\int\pi_\theta(a_t|s_t)\cdot\nabla_\theta\log\pi_\theta(a_t|s_t)da_t\\
> &= \mathbb{E}_{s_t\sim\pi_\theta}\int\nabla_\theta\pi_\theta(a_t|s_t)da_t\\
> &= \mathbb{E}_{s_t\sim\pi_\theta}\nabla_\theta\int\pi_\theta(a_t|s_t)da_t\\
> &= \mathbb{E}_{s_t\sim\pi_\theta}\nabla_\theta 1da_t=0
> \end{aligned}
> $$
>
> 概率密度函数$\int\pi_\theta(a_t|s_t)da_t=1$

为了方便计算将γ^t理解为学习率 α 的一部分，策略梯度可简化为：

$\nabla_\theta J(\theta)=\mathbb{E}_{\tau\sim\pi_\theta}\left[\sum_{t=0}^{T-1}\nabla_\theta\log\pi_\theta(a_t|s_t)\cdot G_t\right]$

进一步，我们可以将期望展开为对状态-动作对 (s_t,a_t) 的期望：

$\nabla_\theta J(\theta)=\mathbb{E}_{s_t,a_t\sim\pi_\theta}\left[\nabla_\theta\log\pi_\theta(a_t|s_t)\cdot G_t\right]$

3.4.6用状态-动作值函数替代回报

由于 G_t 是随机变量，我们可以用其期望——**状态-动作值函数**$Q^\pi(s_t,a_t)$ 来替代，因为：

$Q^\pi(s_t,a_t)=\mathbb{E}\left[G_t\mid s_t,a_t\right]$

因此策略梯度定理的标准形式为：

$\nabla_\theta J(\theta)=\mathbb{E}_{s_t,a_t\sim\pi_\theta}\left[\nabla_\theta\log\pi_\theta(a_t|s_t)\cdot Q^\pi(s_t,a_t)\right]$

3.4.7引入优势函数（降低方差）

为了降低策略梯度估计的方差，我们可以引入**优势函数**$A^\pi(s_t,a_t)=Q^\pi(s_t,a_t)-V^\pi(s_t)$，其中 $V^\pi(s_t)=\mathbb{E}[G_t\mid s_t]$ 是状态值函数。

由于 $\mathbb{E}_{a_t\sim\pi_\theta(a_t|s_t)}\left[\nabla_\theta\log\pi_\theta(a_t|s_t)\cdot V^\pi(s_t)\right]=0$（$V^\pi(s_t)$ 与 a_t 无关，可从期望中提出，而$\mathbb{E}_{a_t\sim\pi_\theta(a_t|s_t)}[\nabla_\theta\log\pi_\theta(a_t|s_t)]=0$）

因此：

$\nabla_\theta J(\theta)=\mathbb{E}_{s_t,a_t\sim\pi_\theta}\left[\nabla_\theta\log\pi_\theta(a_t|s_t)\cdot A^\pi(s_t,a_t)\right]$

使用优势函数替代状态-动作值函数可以显著降低梯度估计的方差，同时保持期望不变。

3.4.8最终结论

策略梯度定理给出了策略参数梯度的精确表达式，其核心形式包括：

1. **基础形式**：　$\nabla_\theta J(\theta)=\mathbb{E}_{\tau\sim\pi_\theta}\left[\sum_{t=0}^{T-1}\nabla_\theta\log\pi_\theta(a_t|s_t)\cdot G_t\right]$
2. **标准形式**：　$\nabla_\theta J(\theta)=\mathbb{E}_{s_t,a_t\sim\pi_\theta}\left[\nabla_\theta\log\pi_\theta(a_t|s_t)\cdot Q^\pi(s_t,a_t)\right]$
3. **方差降低形式**：　$\nabla_\theta J(\theta)=\mathbb{E}_{s_t,a_t\sim\pi_\theta}\left[\nabla_\theta\log\pi_\theta(a_t|s_t)\cdot A^\pi(s_t,a_t)\right]$

这些公式是 REINFORCE、PPO、A2C 等策略优化算法的理论基石，它们通过采样轨迹来估计梯度，并使用梯度上升来更新策略参数。

3.5重要性采样（Importance Sampling）

3.5.1为什么要有重要性采样

在 RL 中，我们的目标是最大化某个策略 π 下的期望回报：

$J(\pi)=\mathbb{E}_{\tau\sim\pi}[R(\tau)]$

问题来了： 如果我们手里的数据 τ 是服从分布 μ 的，但我们要计算和优化的是在分布 π 下的期望。这就必须用到重要性采样，把 $\mathbb{E}_{\tau\sim\pi}$ 转换为 $\mathbb{E}_{\tau\sim\mu}$。

3.5.2轨迹级别的重要性采样推导

$J(\pi)=\int R(\tau)P_\pi(\tau)d\tau=\int R(\tau)\frac{P_\pi(\tau)}{P_\mu(\tau)}P_\mu(\tau)d\tau=\mathbb{E}_{\tau\sim\mu}\left[\frac{P_\pi(\tau)}{P_\mu(\tau)}R(\tau)\right]$

轨迹概率：$P_\pi(\tau)=P(s_0)\prod_{t=0}^{T-1}\pi(a_t|s_t)P(s_{t+1}|s_t,a_t)$

重要性权重（Importance Weight） $\rho(\tau)=\frac{P_\pi(\tau)}{P_\mu(\tau)}=\frac{P(s_0)\prod_{t=0}^{T-1}\pi(a_t|s_t)P(s_{t+1}|s_t,a_t)}{P(s_0)\prod_{t=0}^{T-1}\mu(a_t|s_t)P(s_{t+1}|s_t,a_t)}=\prod_{t=0}^{T-1}\frac{\pi(a_t|s_t)}{\mu(a_t|s_t)}$

$J(\pi)=\mathbb{E}_{\tau\sim\mu}\left[\prod_{t=0}^{T-1}\frac{\pi(a_t|s_t)}{\mu(a_t|s_t)}R(\tau)\right]$

3.5.3致命缺陷：方差爆炸（Variance Explosion）

虽然这个公式在数学期望上是无偏的，但在实际采样计算中会遇到一个极其致命的问题：**方差爆炸。**

- **连乘效应**：重要性权重 $\rho(\tau)$ 是 T 个连乘项构成的。在实际环境中，轨迹长度 T 可能成百上千。
- **数值极不稳定**：如果目标策略 π 和行为策略 μ 哪怕只有一点点差异，经过 T 次连乘后，$\rho(\tau)$ 要么趋近于 0（如果大多步 $\pi<\mu$），要么爆炸到无穷大（如果某些步 $\pi>\mu$ 且分母极小）。
- **结果**：这会导致我们采样算出来的梯度忽大忽小，模型根本无法收敛。

3.5.4引入因果性（Causality）

在轨迹级别的公式中，我们用整个轨迹的权重去乘整个轨迹的总回报 $R(\tau)$。但实际上，时刻 t 的奖励 r_t 只和时刻 t 及之前的状态、动作有关。

因此，我们可以把总回报拆开，将 $R(\tau)=\sum_{t=0}^{T-1}\gamma^t r_t$ 代入，并去掉与当前奖励无关的未来权重。优化后的目标函数可以化简为：

$J(\pi)=\mathbb{E}_{\tau\sim\mu}\left[\sum_{t=0}^{T-1}\gamma^t r_t\left(\prod_{k=0}^{t}\frac{\pi(a_k|s_k)}{\mu(a_k|s_k)}\right)\right]$

但还是无法不免方差过大的问题

3.5.5单步重要性采样

$\prod_{k=0}^{t}\frac{\pi(a_k|s_k)}{\mu(a_k|s_k)}=\left(\prod_{k=0}^{t-1}\frac{\pi(a_k|s_k)}{\mu(a_k|s_k)}\right)\times\frac{\pi(a_t|s_t)}{\mu(a_t|s_t)}$

将前半部分还原

$\prod_{k=0}^{t-1}\frac{\pi(a_k|s_k)}{\mu(a_k|s_k)}=\frac{P(s_0)\prod_{k=0}^{t-1}\pi(a_k|s_k)P(s_{k+1}|s_k,a_k)}{P(s_0)\prod_{k=0}^{t-1}\mu(a_k|s_k)P(s_{k+1}|s_k,a_k)}\approx\frac{d_\pi(s_t)}{d_\mu(s_t)}$

$[P(s_0)\prod_{k=0}^{t-1}\pi(a_k|s_k)P(s_{k+1}|s_k,a_k)]\mid_{s_t}$是$d_\pi(s_t)$的无偏估计

如果π和μ分布相差不大则有$d^\pi(s_t)\approx d^\mu(s_t)$即$\frac{P(s_0)\prod_{k=0}^{t-1}\pi(a_k|s_k)P(s_{k+1}|s_k,a_k)}{P(s_0)\prod_{k=0}^{t-1}\mu(a_k|s_k)P(s_{k+1}|s_k,a_k)}\approx 1$

后果： 这使得算法变成了**有偏估计（Biased Estimator）**，但换取了**低方差。**

3.5.6重要性采样下的策略梯度

策略梯度$\nabla_\theta J(\theta)=\mathbb{E}_{\tau\sim\pi_\theta}\left[\sum_{t=0}^{T-1}\nabla_\theta\log\pi_\theta(a_t|s_t)\cdot G_t\right]=\sum_{t=0}^{T-1}\mathbb{E}_{\tau\sim\pi_\theta}\left[\nabla_\theta\log\pi_\theta(a_t|s_t)\cdot G_t\right]$

加入重要性采样（一般是与π_θ相差很小的$\pi_{\theta_{old}}$）

$$\begin{aligned}\nabla_\theta J(\theta)&=\sum_{t=0}^{T-1}\mathbb{E}_{\tau\sim\pi_{\theta_{old}}}\left[\prod_{k=0}^{T-1}\frac{\pi_\theta(a_k|s_k)}{\pi_{\theta_{old}}(a_k|s_k)}\cdot\nabla_\theta\log\pi_\theta(a_t|s_t)\cdot G_t\right]\\&=\sum_{t=0}^{T-1}\mathbb{E}_{\tau\sim\pi_{\theta_{old}}}\left[\underbrace{\prod_{k=0}^{t-1}\frac{\pi_\theta(a_k|s_k)}{\pi_{\theta_{old}}(a_k|s_k)}}_{\text{第一段：过去}}\cdot\underbrace{\frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}}_{\text{第二段：现在}}\cdot\underbrace{\prod_{k=t+1}^{T-1}\frac{\pi_\theta(a_k|s_k)}{\pi_{\theta_{old}}(a_k|s_k)}\cdot G_t}_{\text{第三段：未来}}\cdot\nabla_\theta\log\pi_\theta(a_t|s_t)\right]\\&=\sum_{t=0}^{T-1}\mathbb{E}_{\tau_{0:t}\sim\pi_{\theta_{old}}}\left[\underbrace{\prod_{k=0}^{t-1}\frac{\pi_\theta(a_k|s_k)}{\pi_{\theta_{old}}(a_k|s_k)}}_{\text{第一段：过去}}\cdot\underbrace{\frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}}_{\text{第二段：现在}}\cdot\nabla_\theta\log\pi_\theta(a_t|s_t)\cdot\underbrace{\mathbb{E}_{\tau_{t+1:T-1}\sim\pi_{\theta_{old}}}\left[\prod_{k=t+1}^{T-1}\frac{\pi_\theta(a_k|s_k)}{\pi_{\theta_{old}}(a_k|s_k)}\cdot G_t\right]\mid_{s_t,a_t}}_{\text{第三段：未来}}\right]\end{aligned}$$

> 重期望公式：$\mathbb{E}_X[X]=\mathbb{E}_Y[\mathbb{E}_X[X\mid Y]]$，$X$ 是整条轨迹 $\tau_{0:T-1}$ 的所有随机性（由于作用于未来$0:t$部分没有什么效果可以舍弃）。Y 是轨迹的前半部分 $\tau_{0:t}$
> 第三段未来部分可以转化为$\mathbb{E}_{\tau_{t+1:T-1}\sim\pi_\theta}[G_t\mid s_t,a_t]=Q^{\pi_\theta}(s_t,a_t)$

根据3.5.5可知$\prod_{k=0}^{t-1}\frac{\pi(a_k|s_k)}{\mu(a_k|s_k)}\approx 1$，并且由于此时中括号内部已经不再依赖 t 时刻之前的具体动作路径（因为连乘被抹掉了），只依赖于当前时刻的 s_t 和 a_t，我们可以把期望的下标从前缀轨迹 $\tau_{0:t}$ 放宽为整条轨迹 $\tau\sim\pi_{\theta_{old}}$（因为包含关系），并将求和符号放进期望里面

$\nabla_\theta J(\theta)=\mathbb{E}_{\tau\sim\pi_{\theta_{old}}}\left[\sum_{t=0}^{T-1}\frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}\nabla_\theta\log\pi_\theta(a_t|s_t)\cdot Q^{\pi_\theta}(s_t,a_t)\right]$

$J_{IS}(\theta) = \mathbb{E}_{\tau\sim\pi_{\theta_{old}}}\left[\sum_{t=0}^{T-1} \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)} \cdot Q^{\pi_\theta}(s_t, a_t)\right]$

3.6广义优势估计（Generalized Advantage Estimation, GAE）

广义优势估计是深度强化学习中用于估计优势函数的一项核心技术，其主要目的是在策略梯度方法中**平衡偏差（Bias）和方差（Variance）**。

3.6.1 优势函数与 TD 误差

优势函数：$A^\pi(s_t, a_t) = Q^\pi(s_t, a_t) - V^\pi(s_t)$

状态价值函数的TD误差：$\delta_t^V = r_t + \gamma V(s_{t+1}) - V(s_t)$

3.6.2 k 步优势估计

单步 TD 误差只考虑了当前的即时奖励 r_t，因此具有较低的方差，但由于严重依赖近似的 $V(s_{t+1})$，它往往存在较高的偏差。为了降低偏差，我们可以将视野扩展到未来 k 步。我们可以定义 k 步优势估计器 $\hat{A}_t^{(k)}$，它由连续 k 步的 TD 误差累加而成：

- **1 步估计:** $\hat{A}_t^{(1)} = \delta_t^V = -V(s_t) + r_t + \gamma V(s_{t+1})$
- **2 步估计:** $\hat{A}_t^{(2)} = \delta_t^V + \gamma\delta_{t+1}^V = -V(s_t) + r_t + \gamma r_{t+1} + \gamma^2 V(s_{t+2})$
- **k 步估计:** $\hat{A}_t^{(k)} = \sum_{l=0}^{k-1} \gamma^l \delta_{t+l}^V = -V(s_t) + r_t + \gamma r_{t+1} + \cdots + \gamma^{k-1} r_{t+k-1} + \gamma^k V(s_{t+k})$

随着 k 的增加，我们在计算中引入了更多的真实奖励信号，对近似价值函数 $V(s)$ 的依赖逐渐推迟到未来。这会**降低偏差，但同时也会增加方差。**

3.6.3 GAE:指数加权移动平均

GAE 的核心思想与 TD(λ) 算法非常相似，它并不是只选择某一个固定的 k，而是对所有可能的 k 步估计器求**指数加权移动平均。**

引入衰减参数 $\lambda \in [0, 1]$，为了保证权重之和为 1，我们乘以因子 $(1-\lambda)$：

$\hat{A}_t^{GAE(\gamma,\lambda)} = (1-\lambda)\sum_{k=1}^{\infty} \lambda^{k-1} \hat{A}_t^{(k)}$

现在我们将前面的 k 步估计器展开并代入公式：

$$\begin{aligned}
\hat{A}_t^{GAE(\gamma,\lambda)} &= (1-\lambda)\left[\hat{A}_t^{(1)} + \lambda\hat{A}_t^{(2)} + \lambda^2\hat{A}_t^{(3)} + \ldots\right] \\
&= (1-\lambda)\left[\delta_t^V + \lambda(\delta_t^V + \gamma\delta_{t+1}^V) + \lambda^2(\delta_t^V + \gamma\delta_{t+1}^V + \gamma^2\delta_{t+2}^V) + \ldots\right] \\
&= \delta_t^V(1-\lambda)(1+\lambda+\lambda^2+\ldots) + \gamma\delta_{t+1}^V(1-\lambda)(\lambda+\lambda^2+\ldots) + \gamma^2\delta_{t+2}^V(1-\lambda)(\lambda^2+\lambda^3+\ldots) + \ldots
\end{aligned}$$

利用等比数列求和公式 $\sum_{i=0}^{\infty}\lambda^i = \frac{1}{1-\lambda}$，我们可以化简前面的系数：

- 对于 δ_t^V 项：$(1-\lambda)\frac{1}{1-\lambda} = 1$
- 对于 $\gamma\delta_{t+1}^V$ 项：$(1-\lambda)\frac{\lambda}{1-\lambda} = \lambda$
- 对于 $\gamma^2\delta_{t+2}^V$ 项：$(1-\lambda)\frac{\lambda^2}{1-\lambda} = \lambda^2$

最终，我们得到了 GAE 非常优雅的计算公式：

$\hat{A}_t^{GAE(\gamma,\lambda)} = \sum_{l=0}^{\infty}(\gamma\lambda)^l\delta_{t+l}^V$

3.6.4 GAE 的两个极端情况分析

这个最终公式清晰地展示了 λ 是如何控制偏差和方差的权衡的：

- **当 $\lambda = 0$ 时:**
  $\hat{A}_t^{GAE(\gamma,0)} = \delta_t^V = r_t + \gamma V(s_{t+1}) - V(s_t)$
  此时退化为单步 TD 误差。
  特点：极低的方差，高偏差（因为严重依赖当前的价值网络 V）。
- **当 $\lambda = 1$ 时:**
  $\hat{A}_t^{GAE(\gamma,1)} = \sum_{l=0}^{\infty}\gamma^l\delta_{t+l}^V = \sum_{l=0}^{\infty}\gamma^l r_{t+l} - V(s_t)$
  此时退化为蒙特卡洛（Monte Carlo）估计（真实的回报减去基线）。所有的项除了 $V(s_t)$ 和最终状态外，中间的 $V(s_{t+i})$ 全部在累加时正负抵消。
  特点：极低的偏差（无偏估计），高方差（因为不同 trajectory 之间的真实奖励总和差异很大）。

在实际的深度强化学习算法开发中（尤其是策略梯度算法组），γ 通常决定了价值的衰减尺度，而 $\lambda \in (0, 1)$（如常用的 0.95）则引入了轻微的偏差，以此换取方差的大幅度降低，从而使得训练过程更加稳定。

第四章：强化学习算法主要划分维度

4.1 按「是否学习环境模型」划分

1. 无模型强化学习（Model-Free RL）

- **核心逻辑**：不建模环境，直接通过与环境交互的**试错数据**，学习「价值函数」或「策略函数」，跳过「环境建模→规划」步骤。
- **特点**：实现简单、鲁棒性强，但**样本效率低**（需要大量交互数据），对复杂环境泛化性一般。
- **代表算法**：Q-learning、SARSA、DQN、PPO、A2C、SAC。

2. 有模型强化学习（Model-based RL）

- **核心逻辑**：先通过交互数据**学习环境模型**（拟合 $P(s'|s,a)$和 $R(s,a,s')$），再基于学到的模型进行「规划」（如动态规划、蒙特卡洛树搜索 MCTS）得到策略。
- **特点**：**样本效率高**（可通过模型虚拟生成数据），但易受「模型误差」影响（模型与真实环境偏差会导致策略失效）。
- **代表算法**：Dyna、MBPO、AlphaGo（结合 MCTS）、PILCO。

> - Model-based = 学世界(学习$P(s'|s,a)$和 $R(s,a,s')$，再用学习到的$P(s'|s,a)$和 $R(s,a,s')$去计算$\pi(a|s)$和$Q(s,a)$)
> - Model-free = 学答案(直接与环境交互学习$\pi(a|s)$和$Q(s,a)$)
> 但:
> - 学世界之后也要算答案
> - 学答案时其实也利用了世界结构

4.2 按「学习对象」划分

1. 基于价值的方法 (Value-based)

这类算法的目标是学习一个**价值函数**。它不直接学习动作，而是学习“在某个状态下采取某个动作到底有多值钱”。

- **核心逻辑**：通过计算每个动作的预期收益（Q值），在做决策时，简单粗暴地选择那个 Q 值最大的动作。
- **代表算法**：Q-Learning, DQN, SARSA。
- **特点**：
  - **优点**：采样效率通常较高，比较稳定。
  - **缺点**：通常只能处理**离散**的动作空间（比如：左转、右转、跳跃）；很难处理连续动作（比如：控制方向盘转动的精确角度）。

2. 基于策略的方法 (Policy-based)

这类算法不通过价值函数“绕弯子”，而是直接学习**策略函数** $\pi(a|s)$。

- **核心逻辑**：它建立一个从状态到动作概率的映射。如果某个动作导致了高奖励，就增加该动作出现的概率。
- **代表算法**：策略梯度, REINFORCE。
- **特点**：
  - **优点**：可以处理**连续**动作空间；具有随机性，能够学习到一些随机策略（比如在博弈中不被对手看穿）。
  - **缺点**：训练过程中方差较大，收敛比较慢，且容易陷入局部最优。

3. 演员-评论家方法 (Actor-Critic)

这是目前主流算法的基石，它结合了前两者的优点，把任务分给了两个角色：

- **Actor (演员)**：负责“干活”。它学习策略，决定在当前状态下该做什么动作。
- **Critic (评论家)**：负责“打分”。它学习价值函数，评估 Actor 做的这个动作好不好。
- **工作机制**：Actor 根据 Critic 的反馈来改进策略，而 Critic 根据环境给出的实际奖励来提高自己的打分准确度。
- **代表算法**：A2C, A3C, PPO , SAC 。
- **特点**：兼顾了处理连续动作的能力和训练的稳定性，是目前工业界和学术界最常用的架构。

4.3 按「策略的一致性」划分

在强化学习中，通常涉及两个角色：

1. **行为策略 (Behavior Policy)**：负责与环境互动，产生轨迹数据（State, Action, Reward）。简单说，就是“**谁在实操**”。
2. **目标策略 (Target Policy)**：算法正在学习、优化并希望最终投入使用的策略。简单说，就是“**谁在进步**”。

> 更新时评估的策略，和生成数据的策略，是否是同一个

1. On-policy (同策略)：

- **定义**：行为策略 = 目标策略。
- **逻辑**：自己一边玩，一边总结经验。学习用的数据必须是当前最新策略产生的。一旦策略更新了，旧的数据就作废了，必须重新采样。（新轨迹，即用即扔，采样=更新）
- **代表算法**：SARSA, TRPO, PPO, A2C。

2. Off-policy (异策略)：

- **定义**：行为策略 $\neq$ 目标策略。
- **逻辑**：可以看别人玩（或者看过去的自己玩）来总结经验。学习用的数据可以是其他策略产生的，也可以是以前旧版本策略留下的“历史记录”。（旧经验，反复利用，采样$\neq$更新）
- **代表算法**：Q-Learning, DQN, DDPG, SAC, TD3。

> **“避坑”小贴士：，**
> 虽然 PPO 是典型的 On-policy 算法，但它在实现时其实会把一批数据反复跑几个 Epoch。从严格定义上讲，它在这一小段时间内其实带有一点点“异策略”的味道，只是因为它通过数学手段（Importance Sampling）保证了分布偏移不严重。

4.4 按「交互方式」划分：

1. 在线强化学习 (Online RL)

这是最传统、最经典的 RL 模式。

- **核心逻辑**：边玩边学。智能体采取一个动作，环境给一个反馈，智能体根据反馈立刻更新自己，然后再去尝试。
- **数据流**：策略 $\pi \to$ 产生动作 $a \to$ 环境反馈 $r, s' \to$ 更新策略 π。
- **特点**：
  - **优点**：实时修正，能探索到新的可能性。
  - **缺点**：**代价昂贵且危险。**比如：你不能为了让无人机学避障，就让它在现实中撞坏 1000 架真机；或者为了优化推荐算法，给用户推 100 万个垃圾视频。
- **典型场景**：游戏 AI（如 AlphaGo）、模拟器中的机器人训练。

2. 离线强化学习 (Offline RL)

这是近年来为了解决“落地难”而大火的研究方向。

- **核心逻辑**：**“翻旧账”式学习。**智能体完全不与环境互动，而是通过一份**已经收集好的、静止的数据集**（通常是人类操作记录、旧系统的日志或随机策略产生的轨迹）来学习一个最优策略。
- **数据流**：历史数据集 $\mathcal{D} \to$ 离线训练算法 $\to$ 产出最优策略 π。
- **特点**：
  - **优点**：**安全、低成本。**不需要真实的物理实体参与，直接利用现有的“大数据”。
  - **缺点**：**外推误差 (Extrapolation Error)。**这是最大的痛点——如果数据集里没有某种情况，智能体可能会产生“迷之自信”，做出非常离谱的决策。
- **典型算法**：CQL (Conservative Q-Learning), IQL (Implicit Q-Learning)。

> **“避坑”小贴士：**
> 很多人容易把 Offline RL 和 Off-policy 搞混：
>
> - **Off-policy** 是指学习的策略和产生的策略不同，但它在训练时通常还是会不断采集新数据放进经验回放池（属于 Online 范畴）。
> - **Offline RL** 则是连“采集新数据”这个动作都省了，完全闭门造车。
>
> 比如 DQN, SAC虽然可以利用旧数据，但它们在标准流程中仍然需要不断采集新数据来填补经验回放池，所以它们通常被归类为 Online RL

4.5 按照「动作空间的性质」划分：

这决定了你的算法输出的是“选择题”还是“填空题”。

1. 离散动作空间 (Discrete)：

- 动作是有限且可数的。比如：向左/向右，下围棋的每一个落点。
- **代表算法**：DQN。

2. 连续动作空间 (Continuous)：

- 动作是一个实数值区间。比如：油门踩下的力度（0% 到 100%），关节旋转的角度。
- **代表算法**：PPO, SAC, DDPG。

3. 混合动作空间 (Hybrid)：

- 混合动作空间在游戏的场景中非常常见。例如，王者荣耀手游的技能施放，选择技能1、技能2、技能3，这是离散动作；当确定一个技能后，选择技能的瞄准角度，这是连续动作。
- **代表算法**：PA-DDPG, PDQN, MPDQN。

4.6 按照「决策的确定性」划分：

这决定了智能体在面对同样情况时，表现是否始终如一。

1. 确定性策略 (Deterministic Policy)：

- 输入状态 s，输出唯一的动作 a。
- 数学表达：$a = \mu(s)$。
- **代表算法**：DQN, DDPG, TD3。

2. 随机性策略 (Stochastic Policy)：

- 输入状态 s，输出的是动作的概率分布。
- 数学表达：$a \sim \pi(a|s)$。
- **好处**：自带探索属性，在博弈中（如剪刀石头布）不会被对手预测。
- **代表算法**：SAC, PPO。

4.7 按照「学习范式」划分：

1、动态规划（Dynamic Programming, DP）

- 动态规划方法假设：环境的状态转移概率P已知，奖励函数R已知，也就是我们拥有一个完整的MDP模型$P(s', r|s, a)$
  在这种情况下，可以通过Bellman Equation进行迭代求解最优策略。
- 特点：收敛性强，理论完备，需要已知模型，状态空间大时不可行。
- **代表算法**：策略迭代，价值迭代。

2、蒙特卡洛（Monte Carlo，MC）

- Monte Carlo 方法通过完整采样轨迹来估计价值函数。不需要环境模型，不使用Bootstrapping。
- 核心公式：$V(s_t) \leftarrow V(s_t) + \alpha(G_t - V(s_t))$
- **代表算法**：First-Visit MC，REINFORCE。

3、时间差分（Temporal-Difference，TD）

- 核心洞察：**自举（Bootstrapping）**：用「自己对未来的价值估计」，去更新「当前的价值」
- TD学习的精髓可以用一句话概括：**"用当前奖励和下一步的估计去更新当前的估计"**
- 核心公式：TD误差（TD Error）$\delta_t = r_{t+1} + \gamma V(s_{t+1}) - V(s_t)$
- **代表算法**：TD(0)，SARSA，Q-Learning。

4、策略梯度（Policy Gradient，PG）

- 直接对策略进行参数化并优化($\pi_\theta(a|s)$)，目标 $J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta}[R(\tau)]$
- 优化方式：$\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta)$
- 直观理解： 如果某个动作带来更高回报，就提高它在该状态下的概率。
- **代表算法**：REINFORCE，A2C，PPO

第五章：强化学习算法

一、传统强化学习算法

传统强化学习算法诞生于深度学习普及之前，核心特点是依赖人工设计特征、状态空间和动作空间多为离散且维度较低，无需复杂的神经网络拟合，通过数学推导和迭代更新实现策略优化。

策略迭代 (Policy Iteration)

策略迭代由**评估**和**改进**两个公式闭环组成。

① 策略评估 (Policy Evaluation)

给定一个策略 π，我们要计算它的真实价值。我们将上面的期望方程转化为迭代式（k 表示迭代步数）：

$V_{k+1}(s) = \sum_{a \in A} \pi(a|s) \sum_{s'} P(s'|s,a)[R(s,a,s') + \gamma V_k(s')]$

> **注**：当 V_{k+1} 与 V_k 的差距小于一个极小值 ϵ 时，我们认为评估完成。

② 策略改进 (Policy Improvement)

基于评估出的 $V_\pi(s)$，利用贪心策略（Greedy Policy）来提取更好的策略 π'：

$\pi'(s) = \text{argmax}_a \sum_{s'} P(s'|s,a)[R(s,a,s') + \gamma V_\pi(s')]$

价值迭代 (Value Iteration)

价值迭代不等待策略评估收敛，它直接将贝尔曼最优方程转化为更新规则。其核心在于 $\max$ 操作：

$V_{k+1}(s) = \max_{a \in A} \sum_{s'} P(s'|s,a)[R(s,a,s') + \gamma V_k(s')]$

这个公式隐含了这样一个逻辑：在当前状态，我只看一眼未来能跳到的最优格子，然后直接更新我当下的身价。

SARSA（State-Action-Reward-State-Action）

算法流程

1. 初始化：创建Q表格（状态空间S×动作空间A），所有$Q(s,a)$初始化为0（或随机小值）；设置学习率α、折扣因子γ、探索率ϵ（$\epsilon - greedy$策略，控制探索与利用的平衡）。
2. 对于每个episode（从环境初始化到终止状态的完整交互过程）：
    1. 初始化当前状态s（从环境初始状态开始）。
    2. 根据当前状态s和$\epsilon - greedy$策略，选择当前动作a（ϵ概率随机选动作，$1-\epsilon$概率选Q(s,a)最大的动作）。
    3. 执行动作a，获得环境反馈的奖励r和下一状态s'。
    4. 根据下一状态s'和$\epsilon - greedy$策略，选择下一动作a'（同执行策略，需实际执行）。
    5. 根据更新公式$Q(s,a) \leftarrow Q(s,a) + \alpha \cdot \delta = Q(s,a) + \alpha[r + \gamma Q(s',a') - Q(s,a)]$，更新$Q(s,a)$的值。
    6. 更新当前状态$s = s'$，当前动作$a = a'$。
    7. 判断s'是否为终止状态：若是，结束当前episode；若否，返回步骤2.4，继续交互。
3. 重复步骤2，直到Q表格收敛（多次episode后，$Q(s,a)$值不再明显变化）。
4. 训练完成后，使用贪心策略做决策。

Q学习（Q-Learning）

算法流程

1. 初始化：与SARSA一致，创建Q表格（初始值0或随机小值），设置α、γ、ϵ。
2. 对于每个episode：
    1. 初始化当前状态s。
    2. 根据s和$\epsilon - greedy$策略（行为策略），选择当前动作a，执行a，获得r和s'。
    3. 根据公式$Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma \max_{a'} Q(s',a') - Q(s,a)]$更新$Q(s,a)$：无需选择“实际执行的下一动作$a'$”，直接计算$s'$下$Q$值的最大值。

4. 更新当前状态$s = s'$，判断s'是否为终止状态：若是，结束当前episode；若否，返回步骤2.2（无需更新动作，下一动作a'由下一状态s'的$\epsilon - greedy$策略重新选择）。
3. 重复步骤2，直到Q表格收敛。
4. 最终目标策略为：对于每个状态s，选择$Q(s,a)$最大的动作a^*

Dyna-Q

核心逻辑
Dyna架构包含两个核心部分——① 无模型学习（类似Q-learning，利用实际经验更新Q值和环境模型）；② 模型规划（利用学习到的环境模型，生成虚拟的状态-动作-奖励-状态(s,a,r,s')经验，再用这些虚拟经验更新Q值）。通过"实际探索+虚拟规划"的双重更新，让Q函数快速收敛到最优。
算法流程

1. 初始化：创建Q表格（初始值0或随机小值）；创建转移模型T（记录$s,a \to s'$的计数）和奖励模型R（记录s,a,s'的平均奖励）；设置α（学习率）、γ（折扣因子）、ϵ（探索率）、规划步数N（每次实际交互后，执行N次虚拟更新）。
2. 对于每个episode：
    1. 初始化当前状态s。
    2. 根据s和$\epsilon - greedy$策略，选择动作a，执行a，获得实际奖励r和实际下一状态s'（实际探索步骤）。
    3. 实际经验更新：
        - 更新转移模型$T(s,a,s') += 1$，更新奖励模型$R(s,a,s')$（计算平均奖励）$R_{new}(s,a,s') = R_{old}(s,a,s') + \frac{1}{T(s,a,s')}[r - R_{old}(s,a,s')]$。
        - 用$Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma \max_{a'} Q(s',a') - Q(s,a)]$公式更新Q(s,a)。
    4. 虚拟规划更新（重复N次）：
        - 随机采样一个已访问过的状态s_v（从T中已记录的s中选择）。
        - 随机采样s_v对应的一个已执行过的动作a_v（从T中s_v对应的a中选择）。
        - 通过转移模型$T(s_v,a_v,\cdot)$，采样虚拟下一状态s'_v（按计数频率采样$P(s'_v|s_v,a_v) = \frac{T(s_v,a_v,s'_v)}{\sum_{s''} T(s_v,a_v,s'')}$）。
        - 通过奖励模型$R(s_v,a_v,s'_v)$，获取虚拟奖励r_v。
        - 用$Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma \max_{a'} Q(s',a') - Q(s,a)]$公式，通过虚拟经验$(s_v,a_v,r_v,s'_v)$更新$Q(s_v,a_v)$。
    5. 更新当前状态$s = s'$，判断s'是否为终止状态：若是，结束当前episode；若否，返回步骤2.2。
3. 重复步骤2，直到Q表格收敛。

变体

Dyna-Q+

Dyna-Q的不足：当环境模型存在偏差（或未探索到某些(s,a)）时，虚拟规划会基于错误模型更新Q值，导致收敛偏差。Dyna-Q+对此优化，核心是"鼓励探索未访问过的(s,a)"

Dyna-Q+ 引入了一个时间补偿机制。它会记录每一个状态-动作对 (s,a) 距离上一次被**真实访问**已经过去了多少个时间步。在**虚拟规划（Planning）**阶段，它不再直接使用模型记录的原始奖励 r_v，而是给这个奖励加一个"奖金"：$r_{bonus} = r_v + \kappa\sqrt{\tau(s,a)}$

- $\tau(s,a)$：表示自上一次在真实环境中对状态 s 执行动作 a 以来，已经经过的时间步数。
- κ：一个很小的正系数（探索权重）。

二、深度强化学习算法（DRL）

深度强化学习（DRL）是深度学习与传统强化学习的结合，核心突破是用深度神经网络（DNN）替代人工设计的特征，拟合价值函数或策略函数，解决高维状态空间的问题。

- 策略的损失一般为目标函数的负数（$L(\theta) = -J(\theta)$）。
- 目标网络：一份和主网络结构完全相同、但参数更新更慢 / 暂时固定的 "副网络"，缓解在训练时提供一段稳定不变的学习目标，解决深度强化学习里 "目标一直在飘" 的问题。一般来说价值学习和异策略策略学习都要用到目标网络。
- 在DRL算法中，价值网络（Critic）的更新通常包含 **"目标价值"**，即使目标值公式里涉及网络参数，这些参数通常 **不参与梯度计算**。例如$L(\phi) = \frac{1}{2}[Q_\phi(s_t,a_t) - y_t]^2$一般来说$y_t = r_t + \gamma Q_\phi(s_{t+1},a_{t+1})$可能包含对应网络参数，但通常人为进行 detach操作。

REINFORCE

算法流程

1. 初始化：随机初始化策略网络 π_θ 的权重。
2. 采样（Rollout）：使用当前策略运行一个完整的回合，记录轨迹：$\tau = (s_0, a_0, r_0, s_1, a_1, r_1, \ldots, s_t, a_t, r_t, \ldots, s_T)$。
3. 计算回报：对于该回合中的每一个时刻 t，计算其后续的总回报 G_t。
4. 梯度更新：
   - 利用梯度上升法更新网络参数：
     $\theta \leftarrow \theta + \alpha \gamma^t G_t \nabla_\theta \log \pi_\theta(a_t|s_t)$
   - 将γ^t吸收到学习率 α 中
     $\theta \leftarrow \theta + \alpha G_t \nabla_\theta \log \pi_\theta(a_t|s_t)$
   - 其中 α 是学习率。
5. 循环：重复步骤2，直到策略收敛。

DDQN（Double Deep Q-Network）

两大核心组件

- 经验回放（Experience Replay）： 建立一个缓冲区 D。智能体不再是“学过就忘”，而是把经历存起来，训练时随机抽取。这打破了数据间的相关性，让训练更稳定。
- 目标网络（Target Network）： 准备两个完全一样的神经网络：
  - 估计网络（Eval Net）： 负责实时更新参数，计算当前估值。
  - 目标网络（Target Net）： 负责计算“理论目标值”。它不参与实时训练，而是隔一段时间从估计网络那里同步一次参数。这解决了“一边跑一边追自己影子”的不稳定问题。

算法流程

1. 初始化
   1. 初始化经验回放池 D，设定其容量上限 N。
   2. 初始化估计网络 Q，随机生成参数 θ。
   3. 初始化目标网络 $\hat{Q}$，令其参数 $\theta^- = \theta$。
2. 对于每个episode：
   1. 观察： 获取环境当前状态 s。
   2. 选择动作： 使用 ϵ-greedy 策略。
      1. 有 ϵ 的概率随机选一个动作 a。
      2. 否则，通过神经网络计算所有动作的 Q 值，选最大的：$a = \arg\max_a Q(s_t, a; \theta)$。
   3. 执行： 执行动作 a，得到奖励 r 和下一个状态 s'。
   4. 存储： 将经验元组 (s, a, r, s') 存入回放池 D。
   5. 学习与更新（核心逻辑）
      1. 随机采样： 从 D 中随机抽取一个小批量B（Mini-batch）的经验数据。
      2. 计算目标值（TD Target）： 对于样本中的每一个数据，计算该动作本应达到的理想 Q 值：
         $y_t = r_t + \gamma \max_{a'} \hat{Q_{\theta^-}}(s_{t+1}, a')$
      3. 注：如果 s_{t+1} 是终止状态，$y_t = r_t$。
      4. 更新网络参数：计算估计值 $Q_\theta(s_t, a_t)$ 与目标值 y_t 之间的损失（通常使用 MSE 或 Huber Loss）：
         $L(\theta) = \mathbb{E}_B\left[(y_t - Q_\theta(s_t, a_t))^2\right]$通过反向传播更新 θ 的值。
3. 同步网络： 每隔固定的步数（比如每 1000 步），将 θ 的值复制给 θ^-，即 $\theta^- \leftarrow \theta$。(或者使用软更新$\theta^- \leftarrow \tau \cdot \theta + (1-\tau) \cdot \theta^-$)
4. 重复2，直到收敛。

> 原生DQN没有Target Net，只有一个Q网络，这种方式存在严重缺陷：目标网络既负责 “选择最优动作”（取 max），又负责 “评估该动作的 Q 值”，导致两个步骤的误差相互叠加，最终使得 Q 值被系统性高估（即预测的 Q 值远高于实际真实 Q 值）

变体

Dueling DQN

在很多游戏或任务场景中，状态本身的好坏（State Value）比具体选哪个动作（Advantage）更重要。原生 DQN：强行让神经网络输出一个综合的 $Q(s,a)$，它很难分辨出这个高分是因为“这一步走得妙”还是“这个位置本来就很爽”。
核心改进
让Q网络分叉成两个支路，输入状态s

- 状态价值支路 (Value Stream)： 输出一个标量 $V(s)$，代表当前状态有多好。
- 优势函数支路 (Advantage Stream)： 输出一个向量 $A(s,a)$，代表每个动作比平均水平好多少。
- **聚合公式**
  - 平均聚合：$Q_\theta(s,a) = V_{\theta'}(s) + \left(A_{\theta''}(s,a) - \frac{1}{|\mathcal{A}|}\sum_{a'} A_{\theta''}(s,a')\right)$
  - 最大聚合：$Q_\theta(s,a) = V_{\theta'}(s) + A_{\theta''}(s,a) - \max_{a'} A_{\theta''}(s,a')$

A2C（Advantage Actor-Critic，优势演员-评论家算法）

A2C 是强化学习中非常优雅且高效的算法，它结合了基于价值（Value-based）和基于策略（Policy-based）两类方法的优点，并通过引入“优势函数（Advantage Function）”极大地降低了策略梯度更新时的方差。

算法流程

1. 初始化：
   1. 初始化 Actor 网络参数θ、Critic 网络参数ϕ。
   2. 设置折扣因子γ、熵权重β、学习率lr等超参数。
2. 收集轨迹数据：
   1. 让 Actor 在环境中运行策略π_θ，收集一批轨迹数据：$s_0, a_0, r_0, s_1, a_1, r_1, \ldots, s_T$；
3. 计算优势和回报：
   1. 对每个时刻t，计算 TD 优势：$A_t = r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t)$；
      （可选）用n步回报估计优势（进一步降低方差）：$A_t^{(n)} = \sum_{k=0}^{n-1} \gamma^k r_{t+k} + \gamma^n V_\phi(s_{t+n}) - V_\phi(s_t)$
4. 计算损失并更新参数：
   1. 计算 Actor 损失：$L_{Actor}(\theta) = \mathbb{E}_t\left[-\log \pi_\theta(a_t|s_t) A_t\right]$、
   2. Critic 损失：$L_{Critic}(\phi) = \mathbb{E}_t[\frac{1}{2}(\underbrace{r_t + \gamma V_\phi(s_{t+1})}_{\text{冻结参数}} - V_\phi(s_t))^2]$
   3. 熵损失：$-\beta \cdot \mathcal{H}(\pi_\theta(\cdot|s_t))$。
5. 迭代训练：重复步骤 2，直到策略收敛（比如平均奖励不再提升）。

> 原生Actor-Critic算法的价值网络直接评估Q函数值，
> $L_{Actor}(\theta) = \mathbb{E}_t[-\log \pi_\theta(a_t|s_t) Q_\phi(s_t,a_t)]$
> $L(\phi) = \mathbb{E}_t(r_t + \gamma Q_{\phi^-}(s_{t+1}, a_{t+1}) - Q_\phi(s_t,a_t))^2$

SAC（Soft Actor-Critic）

SAC是适用于连续动作空间的随机性策略，他的actor网络（$\pi_\theta(\cdot|s_t)$）并不直接输出动作，而是输出动作所满足分布（假设为正态分布）的均值和方差（一个网络两个分支）

- **均值向量** $\mu_\theta(s)$：表示最有可能的动作。
- **标准差向量** $\sigma_\theta(s)$（或对数标准差）：控制动作的随机性（探索程度）

> 如果直接在这个分布中采样那么得到的a就无法传播θ的导数，为了能传播导数，要使用重**参数化技巧(Reparameterization Trick)**

$a \sim \pi_\theta(\cdot|s)$等价于$a = \tanh(\mu_\theta(s) + \sigma_\theta(s)\cdot\epsilon), \quad \epsilon \sim \mathcal{N}(0,1)$

计算动作a的概率密度

们定义 $u = \mu_\theta(s) + \sigma_\theta(s)\cdot\epsilon$。这个 u 是未经过 tanh 压缩的原始值，它服从正态分布 $u \sim \mathcal{N}(\mu, \sigma^2)$。

对于 d 维动作空间，其对数概率密度 $\log p(u|s)$ 为：

$\log p(u|s) = \sum_{i=1}^{d}\left(-\frac{1}{2}\left(\frac{u_i-\mu_i}{\sigma_i}\right)^2 - \log(\sigma_i\sqrt{2\pi})\right)$

由于最终动作是 $a = \tanh(u)$，这本质上是对坐标系进行了一个“挤压”。根据概率论，如果 $a = f(u)$，那么：

$\pi(a|s) = p(u|s)\cdot\left|\frac{da}{du}\right|^{-1}$

取对数后得到：

$\log \pi(a|s) = \log p(u|s) - \sum_{i=1}^{d} \log \left| \frac{da_i}{du_i} \right|$

因为 $\tanh(u)$ 的导数是 $1 - \tanh^2(u)$，所以修正项就是 $\log(1 - \tanh^2(u))$。

将以上两步合并，得到 SAC 中计算对数策略值的核心公式：

$\log \pi_\theta(a|s) = \log p(u|s) - \sum_{i=1}^{d} \log(1 - \tanh^2(u_i) + \delta)$

（其中 δ 是一个极小的常数，如 $1e-6$，防止 $\log(0)$ 报错）

算法流程

1. 初始化
   - 初始化所有网络参数 θ, ϕ_1, ϕ_2。
   - 将 Target 网络的参数对齐：$\phi_1^- \leftarrow \phi_1, \phi_2^- \leftarrow \phi_2$。
   - 初始化经验回放池 $\mathcal{D}$。
2. 环境交互（数据收集）
   1. 观察状态 s_t，从 Actor 网络采样动作 $a_t \sim \pi_\theta(\cdot|s_t)$。
   2. 执行动作，获得 r_t 和 s_{t+1}。
   3. 将五元组 $(s_t, a_t, r_t, s_{t+1}, d)$ 存入回放池 $\mathcal{D}$。
3. 采样与计算目标值（训练开始）

   从回放池中随机抽取一个 Batch 的数据。
   1. **计算下一时刻的动作和熵**：根据当前策略，计算 s_{t+1} 下的新动作 a' 及其对数概率 $\log \pi_\theta(a'|s_{t+1})$。
   2. **计算 Soft Q 目标值**：
   - 利用两个 Target Critic 网络取最小值（Clipped Double-Q）：
   - $y = r + \gamma \left(\min_{i=1,2} Q_{\phi_i^-}(s_{t+1}, a') - \alpha \log \pi_\theta(a'|s_{t+1}) \right)$

   > **注意**：这里的 $-\alpha \log \pi$ 就是“软”更新的精髓，它把未来的预期熵直接加进了价值评估里。

4. 更新 Critic 网络
   - 通过最小化均方误差来同时更新两个 Q 网络：$J(\phi_i) = \mathbb{E}_{(s,a,r,s')\sim\mathcal{D}} \left[\mathbb{E}_{a'\sim\pi_\theta(\cdot|s')} \left[(Q_{\phi_i}(s,a) - y)^2 \right] \right]$
5. 更新 Actor 网络
   - Actor 的目标是让 Q 值最大，同时让熵最大。我们使用重参数化后的动作进行求导：
     $J(\theta) = \mathbb{E}_{s\sim\mathcal{D}, a\sim\pi_\theta} \left[\alpha \log \pi_\theta(a|s) - \min_{i=1,2} Q_{\phi_i}(s,a) \right]$
   - 通过梯度下降更新 θ。
6. 自动调节温度参数 α (可选但推荐)
   - 如果 α 是自动学习的，我们会根据当前策略的熵与目标熵的差距来更新 α。如果熵太低了，就调大 α 鼓励探索。
7. 达到一定步数，更新 Target 网络。重复步骤2。

DDPG（Deep Deterministic Policy Gradient，深度确定性策略梯度）

DDPG 是一种基于 Actor-Critic 架构的**无模型、Off-policy** 深度强化学习算法，专门用于解决**连续动作空间**的问题。它结合了 DQN 的经验回放和目标网络机制。

网络定义

DDPG 包含四个神经网络：

- **当前策略网络 (Actor)**：参数为 θ，输出确定性动作 $a = \mu_\theta(s)$。
- **当前价值网络 (Critic)**：参数为 ϕ，评估状态-动作对的 Q 值 $Q_\phi(s,a)$。
- **目标策略网络 (Target Actor)**：参数为 θ^-，用于计算目标 Q 值的下一步动作。
- **目标价值网络 (Target Critic)**：参数为 ϕ^-，用于计算目标 Q 值。

算法流程

第一步：初始化

初始化当前网络参数 θ 和 ϕ。

将当前网络参数复制给目标网络：$\theta^- \leftarrow \theta$，$\phi^- \leftarrow \phi$。

初始化经验回放池 $\mathcal{D}$。

第二步：与环境交互收集数据

在状态 s_t 下，Actor 网络输出动作，并加入随机噪声 $\mathcal{N}$ 以进行探索：

$a_t = \mu_\theta(s_t) + \mathcal{N}$

执行动作 a_t，获得奖励 r_t 和下一个状态 s_{t+1}，将转移数据 (s_t, a_t, r_t, s_{t+1}) 存入经验回放池 $\mathcal{D}$。

第三步：采样与计算目标值

从 $\mathcal{D}$ 中随机采样一个批次 (Mini-batch) 的数据，大小为 N。

利用目标网络计算目标 Q 值 y_i（如果 s_{t+1} 是终止状态，则仅为 r_t）：

$y_t = r_t + \gamma Q_{\phi^-}(s_{t+1}, \mu_{\theta^-}(s_{t+1}))$

第四步：更新价值网络 (Critic)

通过最小化目标值 y_i 与当前 Q 值预测的均方误差 (MSE) 来更新 ϕ：

$L(\phi) = \mathbb{E}_N(y_i - Q_\phi(s_i, a_i))^2$

第五步：更新策略网络 (Actor)

利用 Critic 网络提供的梯度来指导 Actor 更新，最大化累积期望回报。策略梯度公式通过链式法则计算：

$L(\theta) = \mathbb{E}_N[\nabla_\theta Q_\phi(s_t, a_t)]$

$\nabla_\theta J \approx \frac{1}{N}\sum_{i=1}^{N} \nabla_a Q_\phi(s,a)|_{s=s_i, a=\mu_\theta(s_i)} \nabla_\theta \mu_\theta(s)|_{s=s_i}$

(即：保持 ϕ 不变，调整 θ 以最大化输出动作的 Q 值)

第六步：循环

一定步数更新目标网络

重复第二步

变体

TD3（Twin Delayed Deep Deterministic Policy Gradient，双延迟深度确定性策略梯度）

1. 网络定义的改变：从 4 个变成 6 个

DDPG 有 4 个网络。而 TD3 引入了 **双 Critic 机制**：

- **增加：** 一个额外的当前价值网络 Q_{ϕ_2} 和对应的目标价值网络 $Q_{\phi_2^-}$。
- **目的：** 解决 Q 值的**过高估计**问题。

2. 目标值计算的改变：取极小值

采样与计算目标值”中，DDPG 直接计算下一步的 Q 值。TD3 做了两处重大修改：

- **双路评估：** 它同时咨询两个目标 Critic 网络，并取其中的**最小值**。
- **目标策略平滑（Target Policy Smoothing）：** 在计算目标动作时，不再直接用 $\mu_{\theta^-}(s_{t+1})$，而是给它加上了一个被裁剪过的随机噪声。

新的目标值计算公式变为：

$\tilde{a} = \text{clip}(\mu_{\theta^-}(s_{t+1}) + \text{clip}(\epsilon, -c, c), a_{low}, a_{high})$

$y_t = r_t + \gamma \min_{i=1,2} Q_{\phi_i^-}(s_{t+1}, \tilde{a})$

> **逻辑：** 这样能让价值函数在动作空间中更平滑，防止策略因为某些异常的 Q 值尖峰而误入歧途。

3. 更新频率的改变：延迟更新（Delayed Updates）

DDPG 是同步更新 Critic 和 Actor 的。TD3 认为这太激进了：

- **动作：** 只有在 Critic 网络更新了 d 次（通常 $d = 2$）之后，才更新一次 Actor 网络和所有的目标网络（Target Networks）。
- **目的：** 确保价值函数（Critic）已经足够稳定了，再去指导策略（Actor）改进。如果地基（价值评估）还在晃，房子（策略）就不该往上盖。

PPO（Proximal Policy Optimization，近端策略优化）

- 引入**重要性采样**：提高样本利用率
- 裁剪机制（clip）：限制单次更新幅度，使更新更稳定

算法流程

通常，PPO 会使用两个神经网络：**Actor 网络**（参数为 θ，负责输出动作的概率分布）和 **Critic 网络**（参数为 ϕ，负责评估当前状态的价值）。

初始化：

- 初始化 Actor 网络参数 θ 和 Critic 网络参数 ϕ。
- 设置超参数：裁剪常数 ϵ、折扣因子 γ、GAE 参数 λ、优化迭代次数 K 等。

主循环 (对于每个episode):

Step 1: 数据收集 (Data Collection)

- 将旧策略参数设为当前参数：$\theta_{old} \leftarrow \theta$。
- 智能体使用当前的策略 $\pi_{\theta_{old}}$ 在环境中交互 T 个时间步。
- 记录每一步的轨迹数据：状态 s_t、动作 a_t、奖励 r_t、下一个状态 s_{t+1} 以及旧策略对该动作的输出概率 $\pi_{\theta_{old}}(a_t|s_t)$。

Step 2: 计算回报与优势函数 (Advantage Estimation)

- 利用 Critic 网络预测的价值 $V_\phi(s_t)$，计算优势函数 $\hat{A}_t$。目前最常用的是**广义优势估计（GAE, Generalized Advantage Estimation）**：
  $\delta_t = r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t)$
  $\hat{A}_t = \sum_{l=0}^{T-t-1} (\gamma\lambda)^l \delta_{t+l}$
- 对于收集到的一批数据，计算每一个时间步的目标 $V_t^{target} = \hat{A}_t + V_\phi(s_t)$。冻结其关于$\phi$的参数。

Step 3: 多轮优化 (Multiple Epochs Update)

- 将收集到的这批数据打乱，划分为多个 Mini-batch。
- **重复执行 K 个 Epoch：**
  - **计算 Ratio：** 对于每一个 Mini-batch，将状态输入**最新**的 Actor 网络 π_θ，计算最新动作概率，并得出 $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}$。
  - **计算 Actor Loss：** 计算 Actor 损失，并加入熵奖励（Entropy Bonus）以鼓励探索：
    $Loss_{Actor}(\theta) = -\mathbb{E}_t \left[\min \left(r_t(\theta)\hat{A}_t, \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t \right) \right] - \alpha \cdot S[\pi_\theta](s_t)$
  - **计算 Critic Loss：** 使用均方误差计算 Value 损失：
    $Loss_{Critic}(\phi) = \frac{1}{2}(V_\phi(s_t) - V_t^{target})^2$
  - **反向传播与参数更新：** 将 $Loss_{Actor}$ 和 $Loss_{Critic}$ 相加（如果 Actor 和 Critic 共享底层网络参数），或者分别对 θ 和 ϕ 使用梯度下降/上升算法（如 Adam）进行网络参数更新。

Step 4: 清除历史数据

- 清除缓冲区（Buffer）中当前这批交互数据，因为 PPO 是同策略（On-policy）算法，更新完策略后，旧数据就不能再用了。
- 进入下一次迭代的 Step 1

三、LLM-based强化学习

LLM-based 强化学习（也常简称 RL-LLM、语言驱动强化学习），是**以大语言模型为核心决策主体，将强化学习的目标优化、试错学习机制，与 LLM 的语言理解、世界知识、逻辑推理、泛化能力深度融合的技术范式。**它既包含用 RL 优化 LLM 本身的能力与对齐，也包含用 LLM 解决传统 RL 的核心痛点，实现通用决策能力的突破。

- 在 **RL训练 LLM** 时，通常会保留一个 **reference model（ref）**。这个 **ref 模型一般是初始的 SFT 模型的冻结副本。**其核心作用是：**作为策略更新时的“锚点（anchor）”，限制策略偏移，稳定训练。**避免模型学到一些 **极端策略**：如问什么问题都只会道歉（模板化拒答）。训练时使用KL散度约束：
  $D_{KL}(\pi_\theta || \pi_{ref}) = \sum_{a \in \mathcal{V}} \pi_\theta(a|q, o_{<t}) \ln \frac{\pi_\theta(a|q,o_{<t})}{\pi_{ref}(a|q,o_{<t})}$，$\mathcal{V}$是词汇表,但这样开销太大，一般工程上使用近似。
  表示当前策略 π_θ 与参考策略 π_{ref} 在某个状态 $q, o_{<t}$ 下的差异。

- **动作**：一般来说是纯语言动作：分为序列级动作（一句话$o^{(i)}$），token级动作（一个token $o_t^{(i)}$），对于一个问题往往要采样多次。
- **状态**：简单来说一个被拼接进 prompt 的上下文文本，$s_0 = q$（即用户提问的问题），$s_t^{(i)} = s_{t-1}^{(i)} + o_{t-1}^{(i)}$，把刚生成的 token 直接拼接到上下文末尾，没有任何随机性，100% 确定。（$s_t^{(i)}$等价于$q, o_{<t}^{(i)}$）

- **奖励**：一般LLM-RL的奖励来源：
  - **RLHF（Reinforcement Learning from Human Feedback，基于人类反馈的强化学习）**

    依赖人工标注的偏好数据（A 答案比 B 答案好），先训练一个独立的奖励模型（Reward Model, RM），然后用 RM 给 LLM 产出的答案打分（通常在句子或段落结束时给一个标量奖励）。
  - **RLAIF（Reinforcement Learning from AI Feedback，基于 AI 反馈的强化学习）**

    利用更强大的大模型（如 GPT-4）代替人类进行偏好标注或直接打分，大幅降低人工标注成本。
  - **RLVR（Reinforcement Learning with Verifiable Rewards，基于可验证奖励的强化学习）**

    面向有客观对错标准的问题（如数学、编程）。不需要训练 RM，奖励直接来自规则校验器（Rule-based Checker）。例如，数学题提取最后的结果比对标准答案，代码题看是否能通过单元测试（Unit Test）。若能给与正向奖励，否则可以给与负向或者零奖励。
- **奖励模型（Reward Model，RM）**
  人类很难直接对一个模型生成的回答给出一个绝对的、精确的分数（例如：这个回答是 7.85 分），但人类非常擅长进行比较（例如：回答 A 比回答 B 更好）。因此，奖励模型的本质是学习一个**偏好函数**。
  - 核心假设：奖励模型通常基于统计学中的 Bradley-Terry 模型。该模型假设：用户偏好回答 o_w 胜过 o_l 的概率，可以用两者的奖励差值通过 Sigmoid 函数来表示：

    $P(o_w \succ o_l|q) = \frac{\exp(r_\theta(q,o_w))}{\exp(r_\theta(q,o_w))+\exp(r_\theta(q,o_l))} = \sigma(r_\theta(q,o_w) - r_\theta(q,o_l))$

    其中：q：输入的 Prompt。o_w：人类更喜欢的回答（Winning response）；o_l：人类不那么喜欢的回答（Losing response）；r_θ：带参数 θ 的奖励模型。
  - 数据准备：偏好数据集（Pairwise Data）

    训练奖励模型需要大量的对比数据。

    1. **采样**：对于同一个 Prompt x，利用已微调好的 SFT 模型生成 K 个不同的回答（$K \geq 2$）。
    2. **标注**：人类标注员对这 K 个回答进行排序（Rank）。
    3. **构建数据对**：将排序结果拆解成二元组 (q, o_w, o_l)。如果一个 Prompt 有 4 个回答排序为 $A > B > C > D$，则可以提取出 $C_4^2 = 6$ 个数据对。
  - 损失函数（Loss Function）
    训练 RM 是一个二分类任务的变体。目标是让模型给 o_w 打的分数远高于 o_l。其损失函数定义为：
    $L(\theta) = -E_{(q,o_w,o_l)\sim D}[\log(\sigma(r_\theta(q,o_w) - r_\theta(q,o_l)))]$
    逻辑解释：当 $r_\theta(q,o_w)$ 显著大于 $r_\theta(q,o_l)$ 时，$r_w - r_l$ 为正大值，Sigmoid 趋近于 1，负对数损失趋近于 0。

RLHF-PPO

使用 PPO（Proximal Policy Optimization，近端策略优化）训练大语言模型（LLM）是当前 RLHF（基于人类反馈的强化学习） 流程中的核心环节。其目的是让预训练模型（SFT 之后）的输出更符合人类的偏好。

算法流程

我们当前正在训练的 LLM 叫做 Actor 模型（参数为 θ），此外我们保留一个训练前冻结权重的 Reference 模型（符号为为 π_{ref}）用于防止模型在训练中“跑偏”。

Step 1: 采样与经验收集 (Experience Rollout)

从 Prompt 池中随机抽取一批提示词。Actor 模型根据 Prompt 生成一段回答o（Response）

Reward Model 对整个生成结果给出一个标量分数r

计算每个token的奖励$r_t = \begin{cases} -\beta \log \frac{\pi_{\theta_{old}}(o_t|s_t)}{\pi_{ref}(o_t|s_t)} & t < |o| - 1 \\ r - \beta \log \frac{\pi_{\theta_{old}}(o_t|s_t)}{\pi_{ref}(o_t|s_t)} & t = |o| - 1 \end{cases}$

Step 2: 价值估计与优势计算

在 PPO 中，除了 Actor 模型，我们还需要一个 **Critic 模型**（也叫 Value Model，参数记为 ϕ）。它的作用是预测在当前状态 s_t 也即 $(q, o_{<t})$下，未来能获得的预期总收益 $V_\phi(s_t)$

TD误差：$\delta_t = r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t)$

GAE：$\hat{A}_t = \sum_{l=0}^{|o|-1-t} (\gamma\lambda)^l \delta_{t+l}$

目标价值：$\hat{R}_t = \hat{A}_t + V_\phi(s_t)$

Step 3: 模型更新

$L_{actor} = -\mathbb{E}_{q\sim P(Q), o\sim\pi_{\theta_{old}}(\cdot|q)} \left[\frac{1}{|o|} \sum_{t=0}^{|o|-1} \min\left(\rho_t(\theta)\hat{A}_t, \text{clip}(\rho_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t \right) \right]$

其中：$\rho_t(\theta) = \frac{\pi_\theta(o_t|s_t)}{\pi_{\theta_{old}}(o_t|s_t)}$

$L_{critic} = \mathbb{E}_t \left[\frac{1}{2} (V_\phi(s_t) - \hat{R}_t)^2 \right]$

Step 4: 迭代训练

- 完成多轮（Epochs）的梯度下降更新后，将当前最新的 Actor 模型参数 θ 赋值给 θ_{old}。
- 清空经验池，回到 **Step 1** 重新采样，循环往复，直到模型在验证集上的奖励分数收敛。

DPO（Direct Preference Optimization，直接偏好优化）

在 LLM 对齐领域，DPO (Direct Preference Optimization) 的出现具有里程碑意义。它由斯坦福大学团队在 2023 年提出，核心思想是：既然训练奖励模型（RM）是为了优化策略（Policy），那为什么不直接利用偏好数据来更新策略，而跳过中间的强化学习（PPO）过程？

DPO 的逻辑是：**直接将奖励函数与最优策略建立解析映射，从而把强化学习问题转化为一个简单的二分类监督学习问题。**

1.RL 对齐的目标函数

在 RLHF 中，我们希望最大化奖励，同时加入 KL 散度约束防止模型偏离原始分布太远：

$\max_\pi \mathbb{E}_{q\sim D, o\sim\pi_\theta(\cdot|q)} \left[r(q,o) - \beta D_{KL}(\pi(\cdot|q)||\pi_{ref}(\cdot|q)) \right]$

其中 π_{ref} 是 SFT 后的参考模型，β 是控制约束强度的系数。

2.最优解的解析形式

对于给定的 q，我们的目标是最大化：

$$\begin{aligned}
\mathbb{E}_{o\sim\pi(\cdot|q)} \left[r(q,o) - \beta D_{KL}(\pi(\cdot|q)||\pi_{ref}(\cdot|q)) \right] &= \sum_o \pi(o|q) r(q,o) - \beta \sum_o \pi(o|q) \log \frac{\pi(o|q)}{\pi_{ref}(o|q)} \\
&= \sum_o \pi(o|q) \left[r(q,o) - \beta \log \frac{\pi(o|q)}{\pi_{ref}(o|q)} \right] \\
&= \beta \sum_o \pi(o|q) \left[\frac{1}{\beta} r(q,o) - \log \frac{\pi(o|q)}{\pi_{ref}(o|q)} \right]
\end{aligned}$$

由于 $\pi(o|q)$ 是一个概率分布，它必须满足归一化约束：$\sum_o \pi(o|q) = 1$

构造拉格朗日函数：$\mathcal{L}(\pi, \lambda) = \sum_o \pi(o|q) \left[r(q,o) - \beta \log \frac{\pi(o|q)}{\pi_{ref}(o|q)} \right] + \lambda \left(\sum_o \pi(o|q) - 1 \right)$

求偏导：$\frac{\partial \mathcal{L}}{\partial \pi(o|q)} = r(q,o) - \beta \left(\log \frac{\pi(o|q)}{\pi_{ref}(o|q)} + 1 \right) + \lambda$

令 $\frac{\partial \mathcal{L}}{\partial \pi(o|q)} = 0$有$r(q,o) - \beta \log \frac{\pi(o|q)}{\pi_{ref}(o|q)} - \beta + \lambda = 0$

变换得：$\pi(o|q) = \pi_{ref}(o|q) \exp\left(\frac{1}{\beta} r(q,o) \right) \cdot \exp\left(\frac{\lambda - \beta}{\beta} \right)$

为了方便表示，我们定义一个与 o 无关的项（即归一化因子）：$\frac{1}{Z(q)} = \exp\left(\frac{\lambda - \beta}{\beta} \right)$

因为 $\sum_o \pi(o|q) = 1$，代入上式：$\sum_o \left[\frac{1}{Z(q)} \pi_{ref}(o|q) \exp\left(\frac{1}{\beta} r(q,o) \right) \right] = 1$

解出 $Z(q)$（即配分函数）：$Z(q) = \sum_o \pi_{ref}(o|q) \exp\left(\frac{1}{\beta} r(q,o) \right)$

最优策略 $\pi^*(o|q)$ 的解析解为：$\pi^*(o|q) = \frac{1}{Z(q)} \pi_{ref}(o|q) \exp\left(\frac{1}{\beta} r(q,o) \right)$

3.用策略表达奖励

将上面的公式取对数并移项，我们可以得到**奖励函数 $r(x,y)$ 如何用策略 π 来表达**：

$r(q,o) = \beta \log \frac{\pi^*(o|q)}{\pi_{ref}(o|q)} + \beta \log Z(q)$

4.DPO 损失函数

根据 **Bradley-Terry (BT) 偏好模型**：

$P(o_w \succ o_l | q) = \sigma(r(q,o_w) - r(q,o_l))$

将 $r(q,o)$ 的表达式代入 BT 模型中，$Z(q)$ 项被抵消了，最终得到的 DPO 损失函数为：

$\mathcal{L}_{DPO}(\pi_\theta; \pi_{ref}) = -\mathbb{E}_{(q,o_w,o_l)\sim D} \left[\log \sigma \left(\beta \log \frac{\pi_\theta(o_w|q)}{\pi_{ref}(o_w|q)} - \beta \log \frac{\pi_\theta(o_l|q)}{\pi_{ref}(o_l|q)} \right) \right]$

GRPO（Group Relative Policy Optimization，组相对策略优化）

- 在传统的 PPO 算法中，为了衡量当前动作的好坏，我们需要一个Critic（价值模型）来预测状态价值（State Value，即当前状态下未来能获得多少奖励），从而计算优势函数（Advantage）。
  但这在 LLM 训练中带来了一个致命的工程痛点：显存爆炸。PPO 需要同时在显存中维持四个大模型（Actor 策略模型、Reference 参考模型、Critic 价值模型、Reward 奖励模型）。如果训练一个 70B 的大模型，这种多模型并行的显存开销几乎是不可承受的。

- GRPO 的核心思想就是：干掉 Critic 模型。 它通过对同一个问题采样**一组（Group）**答案，在组内进行横向比较来计算优势函数。

算法流程

我们当前正在训练的 LLM 叫做 Actor 模型（参数为 θ），此外我们保留一个训练前冻结权重的 Reference 模型（符号为为 π_{ref}）用于防止模型在训练中"跑偏"。

Step 1: 组采样（Group Sampling）

对于给定的一个用户问题（Prompt）q，Actor 模型基于当前的旧策略 $\pi_{\theta_{old}}$，进行多次随机采样，生成一组共 G 个不同的回答：$\{o^{(1)}, o^{(2)}, \ldots, o^{(G)}\} \sim \pi_{\theta_{old}}(\cdot|q)$（注意：这里的 G 通常是一个相对较小的常数，比如 4 到 8，取决于显存大小。）

Step 2: 奖励评估（Reward Scoring）

将这 G 个回答送入奖励机制（可以是基于规则的 RLVR 校验器，也可以是训练好的 Reward Model），得到每个回答的绝对奖励分数：$\{r^{(1)}, r^{(2)}, \ldots, r^{(G)}\}$

Step 3: 优势函数计算（Advantage Estimation）

这是 GRPO 的灵魂所在。它不依赖 Critic 模型来估计基线（Baseline），而是直接使用这组奖励的**组内统计量**来进行标准化（通常是 Z-score 标准化）。

计算组内平均奖励 μ 和标准差 σ:

$\mu = \frac{1}{G}\sum_{i=1}^{G} r^{(i)}$

$\sigma = \sqrt{\frac{1}{G}\sum_{i=1}^{G}(r^{(i)} - \mu)^2}$

计算每个回答的相对优势$\hat{A}_i$:

$\hat{A}_t^{(i)} = \hat{A}^{(i)} = \frac{r^{(i)}-\mu}{\sigma+\epsilon}$

直观理解：在这个组里，比平均分高的回答，优势为正，鼓励模型多生成；比平均分低的，优势为负，抑制模型生成。

Step 4: 损失计算与策略更新

$$\mathcal{J}_{GRPO}(\theta) = \mathbb{E}_{q\sim P(Q),\{o^{(i)}\}_{i=1}^{G}\sim\pi_{\theta_{old}}(\cdot|q)}\left\{\frac{1}{G}\sum_{i=1}^{G}\frac{1}{|o^{(i)}|}\sum_{t=0}^{|o^{(i)}|-1}\left[\min\left(\rho_t^{(i)}(\theta)\cdot\hat{A}_t^{(i)}, \text{clip}\left(\rho_t^{(i)}(\theta), 1-\epsilon, 1+\epsilon\right)\hat{A}_t^{(i)}\right) - \beta\cdot\mathbb{D}_{KL}(\pi_\theta\|\pi_{ref})|_t^i\right]\right\}$$

其中：$\rho_t^{(i)}(\theta) = \frac{\pi_\theta(o_t^{(i)}|q,o_{<t}^{(i)})}{\pi_{\theta_{old}}(o_t^{(i)}|q,o_{<t}^{(i)})}$

$\mathbb{D}_{KL}(\pi_\theta\|\pi_{ref})|_t^i = \frac{\pi_{ref}(o_t^{(i)}|q,o_{<t}^{(i)})}{\pi_\theta(o_t^{(i)}|q,o_{<t}^{(i)})} - \log\frac{\pi_{ref}(o_t^{(i)}|q,o_{<t}^{(i)})}{\pi_\theta(o_t^{(i)}|q,o_{<t}^{(i)})} - 1$，这里的公式其实是 KL 散度的一个**近似估计器**（通常称为 Schulman 近似），它的优点是方差更低。

Step 4: 迭代训练

- 完成多轮（Epochs）的梯度下降更新后，将当前最新的 Actor 模型参数 θ 赋值给 θ_{old}。
- 清空经验池，回到 **Step 1** 重新采样，循环往复，直到模型在验证集上的奖励分数收敛。

变体

DAPO（Decoupled Clip and Dynamic sAmpling Policy Optimization，解耦裁剪与动态采样策略优化）

DAPO 完全保留 GRPO **组内相对优势、无价值函数**的核心框架，**针对性解决 GRPO 在长思维链（长 CoT）推理训练中的 5 大致命缺陷**，并做 5 项关键升级：

1、解耦上下裁剪阈值：$\epsilon_{low} = 0.2(\text{不变})$，$\epsilon_{high} = 0.28(\text{放宽})$，给低概率探索 token 更大增长空间，**彻底解决熵崩塌**，保持生成多样性与探索能力。

2、强制约束：**0 < 正确数 < G**，**直接过滤全对 / 全错的无梯度样本**，只保留「有对有错」的有效样本，保证每一步都有有效梯度，收敛更快。

3、直接按**所有 token 全局求和再平均**，长文本贡献更合理，单个 token 的奖惩更公平，**完美适配长 CoT 推理**，训练更稳定。

4、采用**软超长惩罚 + 截断过滤**：设置长度缓冲区间，超长梯度度惩罚，完全截断才给 - 1，**大幅降低奖励噪声**，训练更稳。

5、直接**移除 KL 散度项**，让模型自由学习复杂推理模式，适配长思维链的大分布偏移需求。

$$\mathcal{J}_{DAPO}(\theta) = \mathbb{E}_{(q,a)\sim\mathcal{D},\{o^{(i)}\}_{i=1}^{G}\sim\pi_{\theta_{old}}(\cdot|q)}\left[\frac{1}{\sum_{i=1}^{G}|o^{(i)}|}\sum_{i=1}^{G}\sum_{t=0}^{|o_i|-1}\min\left(\rho_t^{(i)}(\theta)\hat{A}_t^{(i)}, \text{clip}(\rho_t^{(i)}(\theta), 1-\epsilon_{low}, 1+\epsilon_{high})\hat{A}_t^{(i)}\right)\right]$$

$\text{s.t. } 0 < \text{count}(\{o^{(i)} \mid \text{is_equivalent}(a, o^{(i)})\}) < G$，LLM 对同一个问题 q 生成的 G 个回答，只要是「全对」或「全错」（是否与标准答案a等价），这组样本直接被过滤，不参与任何损失 / 梯度计算。

GMPO（Geometric-Mean Policy Optimization，几何均值策略优化）

GRPO 训练过程中存在重要性采样比极端的词元，这意味着模型更新并不可靠。这种不稳定性源于 GRPO 的优化目标对重要性加权奖励的异常值十分敏感，异常值会驱动策略进行激进更新，并进一步放大重要性采样比的方差。

GMPO:一种更稳定的 GRPO 变体。GMPO 不再像GRPO那样优化词元级奖励的算术均值，而是最大化其几何均值

$$|J^*_{\text{GMPO}}(\theta)| = \mathbb{E}_{\substack{q\sim P(Q) \\ \{o^{(i)}\}_{i=1}^G \sim \pi_{\theta_{\text{old}}}(\cdot|q)}} \left[\frac{1}{G} \sum_{i=1}^{G} \left(\prod_{t=0}^{|o^{(i)}|-1} |\rho_t^{(i)}(\theta) \hat{A}_t^{(i)}| \right)^{\frac{1}{|o^{(i)}|}} \right] \leq \mathbb{E}_{\substack{q\sim P(Q) \\ \{o^{(i)}\}_{i=1}^G \sim \pi_{\theta_{\text{old}}}(\cdot|q)}} \left[\frac{1}{G} \sum_{i=1}^{G} \frac{1}{|o^{(i)}|} \sum_{t=0}^{|o^{(i)}|-1} \rho_t^{(i)}(\theta) \hat{A}_t^{(i)} \right] = |J^*_{\text{GRPO}}(\theta)|$$

可推导出，$J^*_{GMPO}(\theta)$的取值范围比$J^*_{GRPO}(\theta)$更窄，更窄的取值范围表明，GMPO 的训练过程中优化目标的方差更低，这可作为策略更新更稳定的依据。与GRPO相比，GMPO对异常值更不敏感，因为几何均值天生比算术均值更具抗异常性。

$$\mathcal{J}_{GMPO}(\pi_\theta) = \mathbb{E}_{\substack{q\sim Q \\ \{o^{(i)}\}_{i=1}^G \sim \pi_{\theta_{\text{old}}}(\cdot|q)}} \left[\frac{1}{G} \sum_{i=1}^{G} \left\{ \prod_{t=0}^{|o^{(i)}|-1} \min \left[\rho_t^{(i)}(\theta) \hat{A}_t^{(i)},\ \text{clip}\left(\rho_t^{(i)}(\theta), \epsilon_{\text{low}}, \epsilon_{\text{high}} \right) \hat{A}_t^{(i)} \right] \right\}^{\frac{1}{|o^{(i)}|}} \cdot \text{sgn}\left(\hat{A}_t^{(i)} \right) \right]$$

$$\rho_t^{(i)}(\theta) = \frac{\pi_\theta(o_t^{(i)}|q, o_{<t}^{(i)})}{\pi_{\theta_{old}}(o_t^{(i)}|q, o_{<t}^{(i)})},\ sgn(x) = \begin{cases} 1 & \text{如果 } x > 0 \\ 0 & \text{如果 } x = 0 \\ -1 & \text{如果 } x < 0 \end{cases}$$

GSPO（Group Sequence Policy Optimization，组序列策略优化）

核心创新是**将优化单元从 token 级升级为序列级**，基于**序列级似然**定义重要性比，执行序列级裁剪、奖励与优化；同时对重要性比做**长度归一化**降低方差，严格遵循重要性采样原理，从根源解决 GRPO token 级采样的高方差缺陷。

$$\mathcal{J}_{GSPO}(\theta) = \mathbb{E}_{q\sim P(Q), \{o^{(i)}\}_{i=1}^G \sim \pi_{\theta_{old}}(\cdot|q)} \left\{ \frac{1}{G} \sum_{i=1}^{G} \left[\min \left(s^{(i)}(\theta) \cdot \hat{A}_t^{(i)}, \text{clip}\left(s^{(i)}(\theta), 1-\epsilon, 1+\epsilon \right) \hat{A}_t^{(i)} \right) \right] \right\}$$

$$s^{(i)}(\theta) = \left[\frac{\pi_\theta(o^{(i)}|q)}{\pi_{\theta_{old}}(o^{(i)}|q)} \right]^{\frac{1}{|o^{(i)}|}} = \exp\left[\frac{1}{|o^{(i)}|} \sum_{t=0}^{|o^{(i)}|-1} \frac{\pi_\theta(o_t^{(i)}|q, o_{<t}^{(i)})}{\pi_{\theta_{old}}(o_t^{(i)}|q, o_{<t}^{(i)})} \right]$$

SAPO（Soft Adaptive Policy Optimization，软自适应策略优化）

SAPO致力于解决大语言模型（尤其是混合专家模型 MoE）在强化学习微调中，由于 token 级别重要性比率方差过大导致的更新不稳定问题。现有的主流方法（如 GRPO 和 GSPO）通常依赖**硬裁剪（Hard Clipping）**来限制策略更新幅度，但这会导致一个两难境地：裁剪太严会丢失有效梯度，裁剪太松则会引入离轨噪声。SAPO 的核心创新点可以总结为两个方面：**温度控制的软门控（Soft Gating）和非对称温度设计（Asymmetric Temperatures）**。我们重点通过论文中的公式来拆解这些创新。

在 GRPO 中，目标函数使用了生硬的 $\text{clip}(\rho_t^{(i)}(\theta), 1-\epsilon, 1+\epsilon)$。这意味着只要 token 的重要性比率（新旧策略的概率比）超出设定区间，梯度就会直接归零（一刀切）。

SAPO 的解决方案： SAPO 引入了一个平滑的、基于 Sigmoid 函数的软门控函数 $f_t^{(i)}(x)$：

$$\mathcal{J}_{SAPO}(\theta) = \mathbb{E}_{q\sim P(Q), \{o^{(i)}\}_{i=1}^G \sim \pi_{\theta_{old}}(\cdot|q)} \left[\frac{1}{G} \sum_{i=1}^{G} \frac{1}{|o^{(i)}|} \sum_{t=0}^{|o^{(i)}|-1} f_t^{(i)}\left(\rho_t^{(i)}(\theta) \right) \cdot \hat{A}_t^{(i)} \right]$$

其中，门控函数定义为：$f_t^{(i)}(x) = \sigma(\tau_t^{(i)}(x-1)) \cdot \frac{4}{\tau_t^{(i)}}$，这里的 σ 是 Sigmoid 函数，$\tau_{i,t}$ 是温度参数，用来控制衰减速率。

$$\tau_t^{(i)} = \begin{cases} \tau_{\text{pos}}, & \text{if } \hat{A}_t^{(i)} > 0 \\ \tau_{\text{neg}}, & \text{otherwise} \end{cases},\ \tau_{\text{neg}} > \tau_{\text{pos}} > 0$$